KOGI

Independent Worker Operating System

Low-Level Software Design Specification

Internal Engineering Reference — v0.1.0

Document Type	Low-Level Software Design Specification (LLSDS)
Version	0.1.0 — Initial Engineering Draft
Companion Docs	Part I Architecture · Part II Module Specs · Platform Requirements (PRS v1.0.0)
Coverage	System topology · DB schemas · Service internals · State machines · Algorithms · Caching · Security · Error handling
Primary Language	Zig (Kernel) · Go (API) · Scala (Analytics) · TypeScript (Web) · Java (Desktop)
Build System	Bazel monorepo — all targets cross-compiled and tested in CI
Target Audience	Platform engineers, backend engineers, infrastructure, security team

Table of Contents

1 System Architecture Overview
2 Deployment Topology
3 Kernel Design (Zig)
4 Database Design — Full Schema Catalog
4.1 User & Account Schema
4.2 Portfolio & Resource Schema
4.3 WBS & Story Schema
4.4 Market System Schema
4.5 Exchange System Schema
4.6 Idea Studio Schema
4.7 Community & Social Schema
4.8 Work & OKR Schema
4.9 Crowdfunding & Capital Schema
4.10 AO Governance Schema
4.11 AI Agent Schema
5 Service Layer Design
5.1 API Gateway (Go/Gin)
5.2 Auth Service
5.3 Portfolio Service
5.4 WBS Service
5.5 Market Service
5.6 Exchange Service
5.7 Studio Service
5.8 Community Service
5.9 Work Service
5.10 AI Agent Service
6 State Machine Specifications
6.1 Story Lifecycle
6.2 Order & Escrow Lifecycle
6.3 Crowdfunding Campaign Lifecycle
6.4 AO Proposal Lifecycle
6.5 Idea Studio Pipeline
7 Algorithm Specifications
7.1 Market Recommendation Engine
7.2 Reputation Scoring
7.3 OKR Auto-Scoring
7.4 Sprint Planning Optimizer
7.5 AI Agent Routing
7.6 Portfolio Health Score
8 Event Catalog — Full Payload Schemas
9 API Design — Request/Response Contracts

10 Caching Architecture
11 Security Design
12 Error Handling & Taxonomy
13 Background Job Design
14 Observability & Logging Design

SECTION 1 · ARCHITECTURE

System Architecture Overview

1.1 Architectural Style

Kogi uses a Modular Monolith-to-Microservices migration path. The initial deployment is a well-structured monolith with strict module boundaries enforced by the Zig kernel's registry. Each module can be extracted as an independent microservice without changing the inter-module contract, because all communication already happens through the Event Bus.

ℹ️ **Core Architectural Decisions**

- Event-driven: all cross-module communication is async via the Event Bus; synchronous RPC only within a module
- Shared-nothing storage: each module owns its own database schema; no cross-schema joins in production queries
- CQRS at module boundary: writes go through the command service; reads served by optimized read models / projections
- Outbox pattern: all events written to an outbox table in the same DB transaction as the domain state change — guaranteed delivery
- Idempotency keys: all mutation endpoints accept an Idempotency-Key header; duplicates return the cached response for 24h

1.2 Service Catalog

Service	Language	Protocol	Port	Database	Scaling Unit
api-gateway	Go 1.22	HTTP/2 + gRPC	8080	None (stateless)	Horizontal — N instances behind LB
auth-service	Go 1.22	gRPC	9001	PostgreSQL 16	Horizontal — sticky session via Redis
portfolio-svc	Go 1.22	gRPC + events	9002	PostgreSQL 16	Horizontal
wbs-service	Go 1.22	gRPC + events	9003	PostgreSQL 16	Horizontal
market-svc	Go 1.22	gRPC + events	9004	PostgreSQL 16 + ES	Horizontal
exchange-svc	Go 1.22	gRPC + events	9005	PostgreSQL 16	Active-Active with fencing
studio-svc	Go 1.22	gRPC + events	9006	PostgreSQL 16	Horizontal

Service	Language	Protocol	Port	Database	Scaling Unit
community-svc	Go 1.22	gRPC + WS	9007	PostgreSQL 16	Horizontal + Pub/Sub sharding
work-svc	Go 1.22	gRPC + events	9008	PostgreSQL 16	Horizontal
fund-svc	Go 1.22	gRPC + events	9009	PostgreSQL 16	Active-Passive (financial)
ao-svc	Go 1.22	gRPC + events	9010	PostgreSQL 16	Horizontal
ai-agent-svc	Go 1.22	gRPC + SSE	9011	PostgreSQL 16	Horizontal (GPU-aware scheduling)
kernel	Zig 0.12	FFI + TCP	9000	In-memory + WAL	Single primary + hot standby
event-bus	NATS 2.10	NATS protocol	4222	JetStream (NATS)	Cluster of 3+ nodes
analytics-svc	Scala 2.13	HTTP + gRPC	8081	ClickHouse + Redis	Horizontal
search-svc	Elasticsearch	HTTP	9200	Elasticsearch 8	Cluster — 3+ data nodes
cdn	CloudFront/nginx	HTTPS	443	S3-compatible	Edge nodes globally
web-client	Angular 17	HTTPS	4200	None (SPA)	CDN-distributed

1.3 Data Flow: Write Path

```
Client Request
  → TLS termination at Load Balancer
  → API Gateway (auth middleware, rate limit, routing)
  → Target Service (validates, applies business rules)
  → Begin DB Transaction {
      UPDATE/INSERT domain tables
      INSERT INTO outbox_events (event_type, payload, status=PENDING)
    }
  → Commit Transaction
  → Return response to client

Background: Outbox Relay Worker (per service)
  → SELECT * FROM outbox_events WHERE status=PENDING LIMIT 100
  → Publish to NATS JetStream topic
  → UPDATE outbox_events SET status=PUBLISHED, published_at=NOW()
  → Retry on NATS failure: exponential backoff, max 10 attempts, then DLQ
```

1.4 Data Flow: Read Path

```
Client Query
  → API Gateway
  → Service Read Handler
  → Check Redis cache (L1): HIT → return cached response
  → Check Read Model / Projection DB (L2): HIT → cache in Redis, return
  → Query primary PostgreSQL (L3): project result, cache in Redis, return

Cache Invalidation: Event-driven
  → Event consumed from NATS
  → Cache invalidation handler resolves affected cache keys
  → DEL or SET with new value in Redis
```

SECTION 2 · INFRASTRUCTURE

Deployment Topology

2.1 Production Topology

Cloud Target	AWS (primary) — GCP-compatible via Terraform modules
Regions	3 active regions: us-east-1 (primary), eu-west-1, ap-southeast-1
Container Runtime	Kubernetes 1.30 — EKS in prod, k3s in dev
Service Mesh	Istio 1.22 — mTLS between all services, traffic policies, circuit breakers
API Gateway Layer	AWS ALB + custom Go gateway — JWT verification, rate limiting, routing
Database	Amazon RDS Aurora PostgreSQL 16 — Multi-AZ per region, read replicas
Cache	ElastiCache Redis 7.2 — Cluster mode, 3 shards × 2 replicas
Message Bus	NATS 2.10 JetStream — 3-node cluster per region, geo-replication
Search	Elasticsearch 8 — 3 data nodes + 1 coordinating node per region
Object Storage	S3 + CloudFront CDN — artifacts, media, exports
Secret Management	AWS Secrets Manager — auto-rotation, versioned
Observability	Prometheus + Grafana + Loki + Tempo (OpenTelemetry traces)

2.2 Kubernetes Namespace Strategy

Namespace	Services	Resource Quota	Network Policy
kogi-core	kernel, auth, api-gateway	CPU: 16c / Mem: 32Gi	Ingress from ingress-nginx only
kogi-domain	portfolio, wbs, market, exchange, studio, community, work, fund, ao	CPU: 64c / Mem: 128Gi	Internal only; egress via service mesh
kogi-ai	ai-agent-svc (GPU nodes)	CPU: 32c / Mem: 64Gi / GPU: 4	Internal only; egress to LLM API endpoints
kogi-data	NATS, Redis, Elasticsearch	CPU: 32c / Mem: 64Gi	kogi-domain + kogi-core egress only
kogi-analytics	analytics-svc, ClickHouse	CPU: 16c / Mem: 32Gi	kogi-domain read-only egress
kogi-cdn	nginx, media-processor	CPU: 8c / Mem: 16Gi	Ingress from internet; egress to S3

2.3 Database Topology

Service	Database Instance	Replication	Backup	Connection Pool
auth-svc	kogi-auth-db	Multi-AZ + 1 read replica	PITR 30d + daily snapshot	PgBouncer — 20 max connections/pod
portfolio-svc	kogi-portfolio-db	Multi-AZ + 2 read replicas	PITR 30d + daily snapshot	PgBouncer — 30 max connections/pod
wbs-svc	kogi-wbs-db	Multi-AZ + 2 read replicas	PITR 30d + daily snapshot	PgBouncer — 30 max connections/pod
market-svc	kogi-market-db	Multi-AZ + 3 read replicas	PITR 30d + daily snapshot	PgBouncer — 40 max connections/pod
exchange-svc	kogi-exchange-db	Multi-AZ — Active/Passive ONLY	PITR 90d + hourly snapshot	PgBouncer — 20 max (financial isolation)
community-svc	kogi-community-db	Multi-AZ + 3 read replicas	PITR 30d + daily snapshot	PgBouncer — 50 max connections/pod
ai-agent-svc	kogi-ai-db	Multi-AZ + 1 read replica	PITR 30d + daily snapshot	PgBouncer — 20 max connections/pod

SECTION 3 · KERNEL

Kernel Design — Zig Implementation

The Kogi Kernel is written in Zig 0.12. It serves as the central orchestrator — managing the module registry, event bus adapter, scheduler, and provisioner. The Kernel exposes a C-compatible FFI interface consumed by the Go API gateway.

3.1 Module Registry — Data Structures

```
// kernel/src/kernel.zig

pub const ModuleStatus = enum { initializing, running, paused, error_state,
stopped };

pub const ModuleDescriptor = struct {
    id:          [36]u8,          // UUID as fixed-size string
    name:        []const u8,
    version:     SemanticVersion,
    status:      ModuleStatus,
    health_url:  []const u8,      // gRPC health check endpoint
    dependencies:[]*ModuleDescriptor,
    start_order: u32,             // topologically sorted startup index
    restart_count: u32,
    last_heartbeat: i64,          // Unix timestamp milliseconds
    error_message: ?[]const u8,
};

pub const Registry = struct {
    modules:   std.HashMap([]const u8, *ModuleDescriptor, StringContext, 80),
    mu:        std.Thread.RwLock,  // reader-writer lock for concurrent access
    allocator: std.mem.Allocator,

    pub fn register(self: *Registry, desc: ModuleDescriptor) !void { ... }
    pub fn deregister(self: *Registry, name: []const u8) !void { ... }
    pub fn getStatus(self: *Registry, name: []const u8) ?ModuleStatus { ... }
    pub fn healthCheck(self: *Registry) []ModuleHealth { ... }
    pub fn startupOrder(self: *Registry) [][]const u8 { ... }  // topo sort
};
```

3.2 Event Bus Adapter

```
// kernel/src/event_bus.zig

pub const EventEnvelope = struct {
```

```
    event_id:       [36]u8,       // UUID v4
    event_type:     []const u8,   // e.g. "portfolio.item.created"
    source_module:  []const u8,
    correlation_id: [36]u8,       // trace correlation across services
    causation_id:   ?[36]u8,      // parent event that caused this one
    payload:        []const u8,   // JSON-encoded payload
    timestamp:      i64,          // Unix nanoseconds
    schema_version: u16,          // for consumer compatibility
    retry_count:    u8,
};

pub const Subscription = struct {
    pattern:   []const u8,        // exact match or "portfolio.*" glob
    handler:   *const fn(*EventEnvelope) anyerror!void,
    consumer_name: []const u8,
    durable:   bool,              // JetStream durable consumer
};

pub const EventBus = struct {
    nats_conn: *NATSConnection,
    subs:      std.ArrayList(Subscription),

    pub fn publish(self: *EventBus, env: EventEnvelope) !void {
        // Write to outbox table FIRST (in caller's tx), then relay
        try self.nats_conn.publish(env.event_type, env.toJSON());
    }
    pub fn subscribe(self: *EventBus, sub: Subscription) !void { ... }
    pub fn matchGlob(pattern: []const u8, subject: []const u8) bool { ... }
};
```

3.3 Scheduler

```
// kernel/src/scheduler.zig

pub const ScheduleEntry = struct {
    id:           [36]u8,
    cron_expr:    []const u8,     // standard 5-field cron
    event_type:   []const u8,     // event to emit on trigger
    payload_tmpl: []const u8,     // JSON template, supports {now} {ts}
    next_run_at:  i64,            // precomputed Unix timestamp
    last_run_at:  ?i64,
    enabled:      bool,
    owner_id:     [36]u8,
};

// Scheduler uses a min-heap keyed on next_run_at for O(log n) scheduling.
// Tick resolution: 1 second. Missed ticks fired immediately on recovery.
pub const Scheduler = struct {
    heap:    std.PriorityQueue(*ScheduleEntry, void, compareNextRun),
    bus:     *EventBus,
    mu:      std.Thread.Mutex,
```

```
    pub fn tick(self: *Scheduler, now: i64) !void {
        while (self.heap.peek()) |entry| {
            if (entry.next_run_at > now) break;
            _ = self.heap.remove();
            try self.bus.publish(buildEvent(entry, now));
            entry.last_run_at = now;
            entry.next_run_at = cronNext(entry.cron_expr, now);
            try self.heap.add(entry);
        }
    }
};
```

3.4 Provisioner — New User Flow

```
// kernel/src/provisioner.zig
// Called by auth-service after email verification.
// All steps execute within a single distributed saga:
// on partial failure, compensating events are emitted.

pub fn provisionUser(user_id: [36]u8, plan: Plan) !void {
    // Step 1: Create account record (auth-service DB)
    try auth_db.insertAccount(user_id, plan);
    errdefer auth_db.deleteAccount(user_id);

    // Step 2: Create workspace
    const ws_id = try workspace_svc.create(user_id, .personal);
    errdefer workspace_svc.delete(ws_id);

    // Step 3: Create root portfolio
    const port_id = try portfolio_svc.createRoot(user_id, ws_id);
    errdefer portfolio_svc.delete(port_id);

    // Step 4: Create exchange wallet
    const wallet_id = try exchange_svc.createWallet(user_id);
    errdefer exchange_svc.deleteWallet(wallet_id);

    // Step 5: Emit provisioning complete event
    try event_bus.publish(.{
        .event_type = "user.provisioned",
        .payload = .{ .user_id=user_id, .workspace_id=ws_id,
                      .portfolio_id=port_id, .wallet_id=wallet_id },
    });
}
```

3.5 FFI Interface (C ABI for Go gateway)

```
// kernel/src/ffi.zig — exported C-compatible interface
export fn kogi_kernel_health() [*:0]const u8 { ... }  // JSON health response
```

```
export fn kogi_module_status(name: [*:0]const u8) u8 { ... }  // ModuleStatus enum
value
export fn kogi_publish_event(envelope_json: [*:0]const u8, len: usize) i32 { ... }
export fn kogi_schedule_add(entry_json: [*:0]const u8, len: usize) i32 { ... }
export fn kogi_schedule_remove(id: [*:0]const u8) i32 { ... }
export fn kogi_provision_user(user_json: [*:0]const u8, len: usize) i32 { ... }
```

SECTION 4 · DATABASE

Database Design — Full Schema Catalog

Schema Conventions

- Primary keys: UUID v4 stored as CHAR(36). No integer sequences in domain tables.
- Timestamps: TIMESTAMPTZ (timezone-aware) for all created_at / updated_at / deleted_at columns.
- Soft deletes: deleted_at TIMESTAMPTZ NULL. Queries always filter WHERE deleted_at IS NULL.
- Optimistic locking: version INTEGER DEFAULT 0 on all mutable entities. Incremented on every UPDATE.
- Outbox: every service DB has an outbox_events table for guaranteed event delivery.
- Audit: every service DB has an audit_log table capturing all write operations.
- Naming: snake_case for all identifiers. Tables singular (user, not users).
- JSON columns: JSONB for flexible metadata. Indexed with GIN where queried.

4.1

User & Account Schema

auth-service database (kogi-auth-db)

Table: account

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Primary key — UUID v4
email	VARCHAR(320)	NOT NULL	—	Unique, lowercased, indexed
email_verified	BOOLEAN	NOT NULL	false	True after email verification flow
password_hash	VARCHAR(256)	NOT NULL	—	Argon2id hash

Column	Type	Null	Default	Description
plan	VARCHAR(20)	NOT NULL	'free'	Enum: free | starter | pro | team | enterprise
status	VARCHAR(20)	NOT NULL	'active'	Enum: active | suspended | deactivated | deleted
mfa_enabled	BOOLEAN	NOT NULL	false	TOTP or SMS MFA enabled
mfa_secret	VARCHAR(128)	NULL	NULL	TOTP secret — encrypted at rest via KMS
timezone	VARCHAR(64)	NOT NULL	'UTC'	IANA timezone string
locale	VARCHAR(10)	NOT NULL	'en-US'	BCP-47 language tag
version	INTEGER	NOT NULL	0	Optimistic lock version
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	Updated by trigger on every write
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft-delete timestamp

Table: profile

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Primary key
account_id	CHAR(36)	NOT	—	FK → account.id, UNIQUE

Column	Type	Null	Default	Description
		NULL		
display_name	VARCHAR(100)	NOT NULL	—	—
handle	VARCHAR(50)	NOT NULL	—	Unique URL-safe handle (e.g. @jordan)
bio	TEXT	NULL	NULL	Max 500 chars
tagline	VARCHAR(160)	NULL	NULL	Short professional tagline
avatar_url	VARCHAR(512)	NULL	NULL	CDN URL to avatar image
location	VARCHAR(100)	NULL	NULL	City, Country
website_url	VARCHAR(512)	NULL	NULL	—
skills	JSONB	NOT NULL	'[]'::jsonb	Array of {name,level,verified_by}
social_links	JSONB	NOT NULL	'[]'::jsonb	Array of {platform,url}
availability_status	VARCHAR(30)	NOT NULL	'available'	Enum: available | busy | not_taking_work | open_to_opportunities
is_public	BOOLEAN	NOT NULL	true	Whether profile visible to non-connections
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT	NOW()	—

Column	Type	Null	Default	Description
		NULL		

Table: session

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Session token identifier (stored in JWT jti claim)
account_id	CHAR(36)	NOT NULL	—	FK → account.id
refresh_token_hash	CHAR(64)	NOT NULL	—	SHA-256 of the refresh token
user_agent	VARCHAR(512)	NULL	NULL	Client user agent string
ip_address	INET	NULL	NULL	Client IP at session creation
expires_at	TIMESTAMPTZ	NOT NULL	—	Access token expiry (24h from creation)
refresh_expires_at	TIMESTAMPTZ	NOT NULL	—	30d refresh token window
revoked_at	TIMESTAMPTZ	NULL	NULL	Set on logout or security revocation
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: auth schema

Index Name	Type	Columns	Purpose
idx_account_email	UNIQUE	account(email)	Login lookup by email
idx_account_status	BTREE	account(status)	Filter active/suspended accounts

Index Name	Type	Columns	Purpose
idx_profile_account	UNIQUE	profile(account_id)	Account-to-profile lookup
idx_profile_handle	UNIQUE	profile(handle)	Public handle lookup
idx_session_account	BTREE	session(account_id)	List sessions per user
idx_session_refresh	BTREE	session(refresh_token_hash)	Refresh token validation
idx_session_expires	BTREE	session(expires_at) WHERE revoked_at IS NULL	Active session expiry sweep

4.2

Portfolio & Resource Schema

portfolio-service database (kogi-portfolio-db)

Table: portfolio

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Primary key
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
owner_type	VARCHAR(10)	NOT NULL	—	Enum: user | ao
name	VARCHAR(200)	NOT NULL	—	—
description	TEXT	NULL	NULL	—
is_root	BOOLEAN	NOT NULL	false	True for the auto-provisioned root portfolio
visibility	VARCHAR(20)	NOT	'private'	Enum: private | team | org | public

Column	Type	Null	Default	Description
		NULL		
tags	JSONB	NOT NULL	'[]'::jsonb	String array
metadata	JSONB	NOT NULL	'{}'::jsonb	Flexible key-value for custom fields
health_score	SMALLINT	NULL	NULL	0–100, computed by AI Agent, refreshed daily
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete

Table: portfolio_item

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Primary key
portfolio_id	CHAR(36)	NOT NULL	—	FK → portfolio.id
parent_id	CHAR(36)	NULL	NULL	FK → portfolio_item.id for nesting
item_type	VARCHAR(30)	NOT NULL	—	Enum: program | project | sub_portfolio | resource | container

Column	Type	Null	Default	Description
name	VARCHAR(200)	NOT NULL	—	—
description	TEXT	NULL	NULL	—
status	VARCHAR(30)	NOT NULL	'active'	Enum: planning | active | on_hold | complete | cancelled
visibility	VARCHAR(20)	NOT NULL	'private'	Enum: private | team | org | public
tags	JSONB	NOT NULL	'[]'::jsonb	String array
metadata	JSONB	NOT NULL	'{}'::jsonb	Type-specific fields (JSON Schema per item_type)
sort_order	FLOAT	NOT NULL	0	Fractional index for ordering within parent
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete

Table: resource (extends portfolio_item)

Column	Type	Null	Default	Description
id	CHAR(36)	NOT	gen_uuid()	PK, also FK → portfolio_item.id

Column	Type	Null	Default	Description
		NULL		
resource_type	VARCHAR(30)	NOT NULL	—	Enum: artifact | asset | capital | land | estate | labor | investment | account
quantity	DECIMAL(18,8)	NULL	NULL	Numeric amount for capital/equity resources
unit	VARCHAR(30)	NULL	NULL	USD | shares | pct | hours | sqft | etc.
valuation	DECIMAL(18,2)	NULL	NULL	Current market value in USD
cost_basis	DECIMAL(18,2)	NULL	NULL	Original cost (for investments)
acquisition_date	DATE	NULL	NULL	—
external_id	VARCHAR(256)	NULL	NULL	Reference to external system (Stripe account ID, etc.)
attachments	JSONB	NOT NULL	'[]'::jsonb	Array of {filename, url, content_type, size_bytes}
resource_meta	JSONB	NOT NULL	'{}'::jsonb	Type-specific fields (depreciation schedule, lease terms, etc.)

Table: portfolio_collaborator

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
portfolio_id	CHAR(36)	NOT NULL	—	FK → portfolio.id
user_id	CHAR(36)	NOT NULL	—	Collaborator user UUID
role	VARCHAR(20)	NOT	—	Enum: owner | admin | member | guest

Column	Type	Null	Default	Description
		NULL		
invited_by	CHAR(36)	NOT NULL	—	FK → account.id
expires_at	TIMESTAMPTZ	NULL	NULL	Null = indefinite
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: portfolio schema

Index Name	Type	Columns	Purpose
idx_portfolio_owner	BTREE	portfolio(owner_id, owner_type)	List portfolios by owner
idx_item_portfolio	BTREE	portfolio_item(portfolio_id) WHERE deleted_at IS NULL	Items within a portfolio
idx_item_parent	BTREE	portfolio_item(parent_id) WHERE deleted_at IS NULL	Hierarchy traversal
idx_item_type_status	BTREE	portfolio_item(item_type, status)	Filter by type and status
idx_item_tags	GIN	portfolio_item(tags)	Tag-based search
idx_item_metadata	GIN	portfolio_item(metadata)	Metadata search
idx_resource_type	BTREE	resource(resource_type)	Resource type filter
idx_collab_portfolio_user	UNIQUE	portfolio_collaborator(portfolio_id, user_id)	Prevent duplicate collaborator

4.3

WBS & Story Schema

wbs-service database (kogi-wbs-db)

Table: wbs

Column	Type	Null	Default	Description
id	CHAR(36)	NOT	gen_uuid()	PK

Column	Type	Null	Default	Description
		NULL		
portfolio_item_id	CHAR(36)	NOT NULL	—	FK → portfolio_item.id (Project or Program), UNIQUE
name	VARCHAR(200)	NOT NULL	—	—
execution_mode	VARCHAR(20)	NOT NULL	'kanban'	Enum: agile | kanban | program | routine | research | ops
current_sprint_id	CHAR(36)	NULL	NULL	FK → sprint.id — active sprint if in agile mode
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: story

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wbs_id	CHAR(36)	NOT NULL	—	FK → wbs.id
parent_id	CHAR(36)	NULL	NULL	FK → story.id (Epic → Story, etc.)
hierarchy_level	SMALLINT	NOT NULL	5	1=WorkPackage 2=Theme 3=Initiative 4=Epic 5=Story 6=Task

Column	Type	Null	Default	Description
story_type	VARCHAR(40)	NOT NULL	—	30+ types: feature | bug | spike | research | etc.
title	VARCHAR(500)	NOT NULL	—	—
description	TEXT	NULL	NULL	Markdown content
acceptance_criteria	TEXT	NULL	NULL	Markdown, required for Done transition
status	VARCHAR(20)	NOT NULL	'backlog'	State machine controlled — see Section 6.1
priority	VARCHAR(10)	NOT NULL	'medium'	Enum: critical | high | medium | low
story_points	SMALLINT	NULL	NULL	—
assignee_id	CHAR(36)	NULL	NULL	User UUID
executor_type	VARCHAR(20)	NOT NULL	'human'	Enum: ai_agent | human | hybrid | machine | custom
executor_config	JSONB	NOT NULL	'{}'::jsonb	Executor-specific settings
sprint_id	CHAR(36)	NULL	NULL	FK → sprint.id
labels	JSONB	NOT NULL	'[]'::jsonb	String array
linked_stories	JSONB	NOT NULL	'[]'::jsonb	Array of {story_id, link_type: blocks|relates|duplicates}
linked_resource_ids	JSONB	NOT NULL	'[]'::jsonb	Portfolio resource refs

Column	Type	Null	Default	Description
due_date	DATE	NULL	NULL	—
completed_at	TIMESTAMPTZ	NULL	NULL	Set on Done transition
ai_summary	TEXT	NULL	NULL	AI-generated summary, refreshed on description change
sort_order	FLOAT	NOT NULL	0	Fractional index for ordering within parent
version	INTEGER	NOT NULL	0	Optimistic lock
created_by	CHAR(36)	NOT NULL	—	FK → account.id
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete

Table: sprint

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wbs_id	CHAR(36)	NOT NULL	—	FK → wbs.id
sprint_number	SMALLINT	NOT NULL	—	Auto-incremented per WBS

Column	Type	Null	Default	Description
goal	VARCHAR(500)	NULL	NULL	Sprint goal statement
status	VARCHAR(20)	NOT NULL	'planning'	Enum: planning | active | review | closed
start_date	DATE	NOT NULL	—	—
end_date	DATE	NOT NULL	—	—
planned_points	SMALLINT	NOT NULL	0	—
completed_points	SMALLINT	NOT NULL	0	Updated by trigger on story Done transition
velocity	DECIMAL(5,2)	NULL	NULL	completed_points / planned_points at close
retro_story_id	CHAR(36)	NULL	NULL	FK → story.id (Review type)
ai_plan_used	BOOLEAN	NOT NULL	false	Whether AI sprint plan was accepted
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
closed_at	TIMESTAMPTZ	NULL	NULL	—

Table: story_status_history

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK

Column	Type	Null	Default	Description
story_id	CHAR(36)	NOT NULL	—	FK → story.id
from_status	VARCHAR(20)	NOT NULL	—	—
to_status	VARCHAR(20)	NOT NULL	—	—
actor_id	CHAR(36)	NOT NULL	—	User or AI Agent UUID
actor_type	VARCHAR(20)	NOT NULL	—	Enum: user | ai_agent | system
note	TEXT	NULL	NULL	Optional comment on transition
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: WBS schema

Index Name	Type	Columns	Purpose
idx_story_wbs_status	BTREE	story(wbs_id, status) WHERE deleted_at IS NULL	Board queries
idx_story_parent	BTREE	story(parent_id) WHERE deleted_at IS NULL	Hierarchy traversal
idx_story_sprint	BTREE	story(sprint_id) WHERE sprint_id IS NOT NULL	Sprint story list
idx_story_assignee	BTREE	story(assignee_id, status)	Assigned stories per user
idx_story_due_date	BTREE	story(due_date) WHERE due_date IS NOT NULL	Deadline sweep
idx_story_type	BTREE	story(story_type, status)	Type-based filtering
idx_story_labels	GIN	story(labels)	Label-based search
idx_story_links	GIN	story(linked_stories)	Dependency graph traversal
idx_sprint_wbs_status	BTREE	sprint(wbs_id, status)	Active sprint lookup

Index Name	Type	Columns	Purpose
idx_history_story	BTREE	story_status_history(story_id, created_at)	Status timeline

4.4

Market System Schema

market-service database (kogi-market-db)

Table: listing

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
provider_id	CHAR(36)	NOT NULL	—	User or AO UUID
provider_type	VARCHAR(10)	NOT NULL	—	Enum: user | ao
listing_type	VARCHAR(30)	NOT NULL	—	Enum: service | gig | product_digital | product_physical | talent | tool | resource_rental | opportunity
title	VARCHAR(300)	NOT NULL	—	Elasticsearch indexed
description	TEXT	NOT NULL	—	Rich text, ES indexed
category	VARCHAR(60)	NOT NULL	—	Top-level category slug
subcategory	VARCHAR(60)	NULL	NULL	—
tags	JSONB	NOT	'[]'::jsonb	ES indexed

Column	Type	Null	Default	Description
		NULL		
pricing	JSONB	NOT NULL	—	{type: hourly|fixed|retainer|free, amount, currency, billing_period}
location_type	VARCHAR(20)	NOT NULL	'remote'	Enum: remote | onsite | hybrid
location_geo	GEOGRAPHY(POINT,4326)	NULL	NULL	PostGIS point for geo-radius search
availability	JSONB	NOT NULL	—	{status: available|busy|unavailable, lead_time_days, capacity_per_week}
media	JSONB	NOT NULL	'[]'::jsonb	Array of {url, type, caption}
credentials	JSONB	NOT NULL	'[]'::jsonb	Array of verified credential refs
status	VARCHAR(20)	NOT NULL	'active'	Enum: active | paused | closed | draft
view_count	INTEGER	NOT NULL	0	—
save_count	INTEGER	NOT NULL	0	—
order_count	INTEGER	NOT NULL	0	—
avg_rating	DECIMAL(3,2)	NOT NULL	0	Materialized from review aggregation
review_count	INTEGER	NOT	0	—

Column	Type	Null	Default	Description
		NULL		
es_doc_id	VARCHAR(64)	NULL	NULL	Elasticsearch document ID for sync
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete

Table: market_order

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
listing_id	CHAR(36)	NOT NULL	—	FK → listing.id
buyer_id	CHAR(36)	NOT NULL	—	User UUID
seller_id	CHAR(36)	NOT NULL	—	User or AO UUID
status	VARCHAR(20)	NOT NULL	'pending'	State machine — see Section 6.2
brief	TEXT	NULL	NULL	Buyer's project brief

Column	Type	Null	Default	Description
proposal	TEXT	NULL	NULL	Seller's accepted proposal
amount	DECIMAL(18,2)	NOT NULL	—	In USD
currency	CHAR(3)	NOT NULL	'USD'	ISO 4217
escrow_id	CHAR(36)	NULL	NULL	FK → exchange.escrow.id (cross-service ref)
portfolio_item_id	CHAR(36)	NULL	NULL	Auto-created Portfolio item in buyer's workspace
wbs_story_id	CHAR(36)	NULL	NULL	Auto-created WBS story in buyer's active WBS
contract_artifact_id	CHAR(36)	NULL	NULL	Signed contract artifact reference
delivery_at	TIMESTAMPTZ	NULL	NULL	Delivery submitted timestamp
completed_at	TIMESTAMPTZ	NULL	NULL	Buyer approval timestamp
review_deadline	TIMESTAMPTZ	NULL	NULL	Auto-release time if buyer doesn't respond
idempotency_key	VARCHAR(64)	NULL	NULL	Unique order creation key
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: review

Column	Type	Null	Default	Description
id	CHAR(36)	NOT	gen_uuid()	PK

Column	Type	Null	Default	Description
		NULL		
order_id	CHAR(36)	NOT NULL	—	FK → market_order.id, UNIQUE per reviewer
reviewer_id	CHAR(36)	NOT NULL	—	FK → account.id
reviewee_id	CHAR(36)	NOT NULL	—	FK → account.id or AO
rating	DECIMAL(2,1)	NOT NULL	—	1.0 – 5.0, 0.5 increments
quality_rating	DECIMAL(2,1)	NOT NULL	—	—
communication_rating	DECIMAL(2,1)	NOT NULL	—	—
timeliness_rating	DECIMAL(2,1)	NOT NULL	—	—
value_rating	DECIMAL(2,1)	NOT NULL	—	—
body	TEXT	NULL	NULL	Max 2000 chars
response	TEXT	NULL	NULL	Provider public response, max 1000 chars
is_verified	BOOLEAN	NOT NULL	true	False if review pattern flagged for moderation
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

4.5

Exchange System Schema

exchange-service database (kogi-exchange-db)

Table: wallet

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID, UNIQUE with owner_type
owner_type	VARCHAR(10)	NOT NULL	—	Enum: user | ao
fiat_available	DECIMAL(18,2)	NOT NULL	0.00	Spendable balance in USD
fiat_reserved	DECIMAL(18,2)	NOT NULL	0.00	Held in active outbound escrows
fiat_pending	DECIMAL(18,2)	NOT NULL	0.00	Inbound pending (not yet settled)
credit_balance	INTEGER	NOT NULL	0	Service credits (integer units)
currency	CHAR(3)	NOT NULL	'USD'	Primary currency for this wallet
stripe_customer_id	VARCHAR(100)	NULL	NULL	Stripe customer reference
stripe_account_id	VARCHAR(100)	NULL	NULL	Stripe Connect account for payouts
kyc_status	VARCHAR(20)	NOT	'pending'	Enum: pending | verified | rejected | required

Column	Type	Null	Default	Description
		NULL		
kyc_verified_at	TIMESTAMPTZ	NULL	NULL	—
version	INTEGER	NOT NULL	0	Optimistic lock — CRITICAL for financial correctness
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: ledger_entry ← IMMUTABLE, NO UPDATES, NO DELETES

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wallet_id	CHAR(36)	NOT NULL	—	FK → wallet.id
entry_type	VARCHAR(30)	NOT NULL	—	Enum: credit | debit | hold | hold_release | credit_award | distribution
amount	DECIMAL(18,2)	NOT NULL	—	Always positive; entry_type indicates direction
currency	CHAR(3)	NOT NULL	—	ISO 4217
balance_after	DECIMAL(18,2)	NOT NULL	—	Running balance snapshot at time of entry
reference_id	CHAR(36)	NOT	—	FK to escrow.id, distribution.id, etc.

Column	Type	Null	Default	Description
		NULL		
reference_type	VARCHAR(30)	NOT NULL	—	Enum: escrow | distribution | deposit | withdrawal | credit_award | fee
description	VARCHAR(500)	NOT NULL	—	Human-readable description
metadata	JSONB	NOT NULL	'{}'::jsonb	Additional context
created_at	TIMESTAMPTZ	NOT NULL	NOW()	Immutable — never updated
created_by	CHAR(36)	NOT NULL	—	Service account UUID

> ⚠️ **Ledger Integrity Rule**
> The ledger_entry table has a row-level trigger that RAISES EXCEPTION on any UPDATE or DELETE. All corrections are compensating entries (new rows). This is enforced at the DB layer regardless of application code.

Table: escrow

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
buyer_id	CHAR(36)	NOT NULL	—	FK → account.id
seller_id	CHAR(36)	NOT NULL	—	FK → account.id or AO

Column	Type	Null	Default	Description
status	VARCHAR(20)	NOT NULL	'created'	State machine — see Section 6.2
amount	DECIMAL(18,2)	NOT NULL	—	Total escrow amount in USD
currency	CHAR(3)	NOT NULL	'USD'	—
funded_at	TIMESTAMPTZ	NULL	NULL	—
released_at	TIMESTAMPTZ	NULL	NULL	—
refunded_at	TIMESTAMPTZ	NULL	NULL	—
review_deadline	TIMESTAMPTZ	NULL	NULL	Auto-release trigger time
conditions	JSONB	NOT NULL	'[]'::jsonb	Array of release conditions
milestones	JSONB	NOT NULL	'[]'::jsonb	Array of {amount, description, released_at}
dispute_id	CHAR(36)	NULL	NULL	FK → dispute.id if disputed
platform_fee	DECIMAL(18,2)	NOT NULL	0.00	Platform fee withheld on release
fee_pct	DECIMAL(5,4)	NOT NULL	0.0250	Fee percentage at time of escrow creation
reference_id	CHAR(36)	NOT NULL	—	order_id or contract_id
reference_type	VARCHAR(30)	NOT NULL	—	Enum: market_order | contract | campaign

Column	Type	Null	Default	Description
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: exchange schema

Index Name	Type	Columns	Purpose
idx_wallet_owner	UNIQUE	wallet(owner_id, owner_type)	Owner-to-wallet lookup
idx_ledger_wallet_ts	BTREE	ledger_entry(wallet_id, created_at)	Transaction history
idx_ledger_reference	BTREE	ledger_entry(reference_id, reference_type)	Reference lookup
idx_escrow_buyer	BTREE	escrow(buyer_id, status)	Buyer's active escrows
idx_escrow_seller	BTREE	escrow(seller_id, status)	Seller's pending receipts
idx_escrow_deadline	BTREE	escrow(review_deadline) WHERE status='delivery_submitted'	Auto-release sweep
idx_escrow_reference	BTREE	escrow(reference_id, reference_type)	Order/contract lookup

4.6

Idea Studio Schema

studio-service database (kogi-studio-db)

Table: idea

Column	Type	Null	Default	Description
id	CHAR(36)	NOT	gen_uuid()	PK

Column	Type	Null	Default	Description
		NULL		
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
owner_id	CHAR(36)	NOT NULL	—	FK → account.id
stage	VARCHAR(20)	NOT NULL	'spark'	Enum: spark | concept | research | prototype | design | parked | promoted
title	VARCHAR(300)	NOT NULL	—	—
raw_capture	TEXT	NOT NULL	—	Original input verbatim
capture_modality	VARCHAR(20)	NOT NULL	—	Enum: text | voice | image | url | email | clip
capture_url	VARCHAR(512)	NULL	NULL	Source URL for url/clip captures
capture_media_url	VARCHAR(512)	NULL	NULL	CDN URL for voice/image captures
problem_statement	TEXT	NULL	NULL	Defined at Concept stage
target_audience	JSONB	NOT NULL	'[]'::jsonb	Array of AudienceProfile
value_prop	JSONB	NOT NULL	'{}'::jsonb	Value Proposition Canvas JSON
assumptions	JSONB	NOT NULL	'[]'::jsonb	Array of {text, is_validated, validation_notes}
viability_score	JSONB	NULL	NULL	{market_size, novelty, feasibility, revenue, timing, overall}

Column	Type	Null	Default	Description
ai_synthesis	JSONB	NULL	NULL	Full AI synthesis output
frameworks	JSONB	NOT NULL	'{}'::jsonb	Completed framework JSONs keyed by framework slug
ip_timestamp_hash	CHAR(64)	NULL	NULL	SHA-256 of title+description+owner_id+timestamp
ip_timestamped_at	TIMESTAMPTZ	NULL	NULL	—
promoted_portfolio_item_id	CHAR(36)	NULL	NULL	FK to Portfolio item on promotion
collaborators	JSONB	NOT NULL	'[]'::jsonb	Array of {user_id, role: view|comment|edit}
tags	JSONB	NOT NULL	'[]'::jsonb	—
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

4.7

Community & Social Schema

community-service database (kogi-community-db)

Table: space

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
name	VARCHAR(100)	NOT NULL	—	UNIQUE — platform-wide
slug	VARCHAR(100)	NOT NULL	—	UNIQUE URL-safe slug
space_type	VARCHAR(20)	NOT NULL	—	Enum: public | private | ao | project | industry | learning
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
description	TEXT	NULL	NULL	—
avatar_url	VARCHAR(512)	NULL	NULL	—
banner_url	VARCHAR(512)	NULL	NULL	—
tags	JSONB	NOT NULL	'[]'::jsonb	—
membership_config	JSONB	NOT NULL	—	{type: open|invite|application|ao_governed, min_reputation}
rules	JSONB	NOT NULL	'{}'::jsonb	Content rules JSON
member_count	INTEGER	NOT NULL	0	Denormalized counter, refreshed by trigger
is_verified	BOOLEAN	NOT NULL	false	Platform-verified Space

Column	Type	Null	Default	Description
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: message

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
room_id	CHAR(36)	NULL	NULL	FK → room.id, NULL for DMs
dm_thread_id	CHAR(36)	NULL	NULL	FK → dm_thread.id, NULL for room messages
author_id	CHAR(36)	NOT NULL	—	User UUID or AI Agent UUID
author_type	VARCHAR(20)	NOT NULL	'user'	Enum: user | ai_agent | system
content_type	VARCHAR(20)	NOT NULL	'text'	Enum: text | rich_text | system | ai_response
content	TEXT	NOT NULL	—	Markdown or plain text; max 10,000 chars
attachments	JSONB	NOT NULL	'[]'::jsonb	Array of {url, filename, content_type, size_bytes, portfolio_ref?}

Column	Type	Null	Default	Description
reactions	JSONB	NOT NULL	'[]'::jsonb	Array of {emoji, user_ids[]}
parent_id	CHAR(36)	NULL	NULL	FK → message.id for thread replies
thread_reply_count	INTEGER	NOT NULL	0	Denormalized counter
mentions	JSONB	NOT NULL	'[]'::jsonb	Array of {user_id, type: user|ai|channel}
is_pinned	BOOLEAN	NOT NULL	false	—
is_deleted	BOOLEAN	NOT NULL	false	Soft delete — content replaced with tombstone
edited_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: community schema (message — high-write table)

Index Name	Type	Columns	Purpose
idx_message_room_ts	BTREE	message(room_id, created_at DESC) WHERE is_deleted=false	Room message pagination
idx_message_dm_ts	BTREE	message(dm_thread_id, created_at DESC) WHERE is_deleted=false	DM thread pagination
idx_message_parent	BTREE	message(parent_id) WHERE parent_id IS NOT NULL	Thread replies
idx_message_author	BTREE	message(author_id, created_at DESC)	User message history
idx_message_mentions	GIN	message(mentions)	Mention notification fan-out

Index Name	Type	Columns	Purpose
idx_space_slug	UNIQUE	space(slug)	Space URL lookup
idx_space_type_tags	GIN	space(tags)	Space discovery by tag

4.8

Work & OKR Schema

work-service database (kogi-work-db)

Table: okr

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User, Team, Cooperative, or AO UUID
owner_type	VARCHAR(20)	NOT NULL	—	Enum: personal | team | cooperative | ao
parent_okr_id	CHAR(36)	NULL	NULL	FK → okr.id for cascade
title	VARCHAR(300)	NOT NULL	—	Objective statement
period_type	VARCHAR(10)	NOT NULL	—	Enum: quarterly | annual
period_year	SMALLINT	NOT NULL	—	—
period_q	SMALLINT	NULL	NULL	1–4, NULL for annual
status	VARCHAR(20)	NOT	'draft'	Enum: draft | active | on_track | at_risk | off_track | complete

Column	Type	Null	Default	Description
		NULL		
score	DECIMAL(3,2)	NULL	NULL	0.00–1.00, computed at period end
key_results	JSONB	NOT NULL	'[]'::jsonb	Array of KeyResult objects (see below)
check_ins	JSONB	NOT NULL	'[]'::jsonb	Array of {date, notes, kr_updates[]}
linked_story_ids	JSONB	NOT NULL	'[]'::jsonb	WBS story IDs contributing to this OKR
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

KeyResult JSON shape: {id, title, current_value, target_value, unit, owner_id, score?: float}

Table: automation_rule

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User, Team, or AO UUID
name	VARCHAR(200)	NOT NULL	—	—

Column	Type	Null	Default	Description
trigger	JSONB	NOT NULL	—	{event_type, conditions[{field, op, value}]}
actions	JSONB	NOT NULL	—	Array of {action_type, params}
executor	VARCHAR(20)	NOT NULL	'ai_agent'	Enum: ai_agent | workflow | human | system
enabled	BOOLEAN	NOT NULL	true	—
execution_count	INTEGER	NOT NULL	0	—
last_triggered_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

4.9

Crowdfunding & Capital Pool Schema

fund-service database (kogi-fund-db)

Table: campaign

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK

Column	Type	Null	Default	Description
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
portfolio_item_id	CHAR(36)	NULL	NULL	FK → portfolio_item.id (links to product/idea)
campaign_type	VARCHAR(30)	NOT NULL	—	Enum: donation | reward | equity | revenue_share | coop_membership
title	VARCHAR(300)	NOT NULL	—	—
description	TEXT	NOT NULL	—	—
goal_amount	DECIMAL(18,2)	NOT NULL	—	In USD
raised_amount	DECIMAL(18,2)	NOT NULL	0.00	Running total, updated by trigger
backer_count	INTEGER	NOT NULL	0	Denormalized, updated by trigger
model	VARCHAR(30)	NOT NULL	'all_or_nothing'	Enum: all_or_nothing | keep_what_you_raise
status	VARCHAR(20)	NOT NULL	'draft'	State machine — see Section 6.3
start_date	DATE	NULL	NULL	—
end_date	DATE	NULL	NULL	—
tiers	JSONB	NOT NULL	'[]'::jsonb	Array of {id, title, amount, description, limit?, remaining?}

Column	Type	Null	Default	Description
equity_pct_offered	DECIMAL(5,2)	NULL	NULL	For equity campaigns
revenue_share_pct	DECIMAL(5,2)	NULL	NULL	For revenue share campaigns
escrow_id	CHAR(36)	NULL	NULL	Exchange escrow for campaign funds
funded_at	TIMESTAMPTZ	NULL	NULL	—
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: capital_pool

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → ao.id, UNIQUE per AO (or UNIQUE per cooperative)
name	VARCHAR(200)	NOT NULL	—	—
balance	DECIMAL(18,2)	NOT NULL	0.00	Current balance in USD
currency	CHAR(3)	NOT NULL	'USD'	—

Column	Type	Null	Default	Description
sub_allocations	JSONB	NOT NULL	'[]'::jsonb	Array of {name, budget, spent}
withdrawal_threshold	DECIMAL(18,2)	NOT NULL	1000.00	Amount above which governance vote required
version	INTEGER	NOT NULL	0	Optimistic lock — financial
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

4.10

AO Governance Schema

ao-service database (kogi-ao-db)

Table: autonomous_org

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
name	VARCHAR(200)	NOT NULL	—	—
slug	VARCHAR(100)	NOT NULL	—	UNIQUE URL slug
org_type	VARCHAR(30)	NOT NULL	—	Enum: cooperative | investment_club | open_source | nonprofit | general

Column	Type	Null	Default	Description
mission	TEXT	NULL	NULL	—
governance_model	VARCHAR(30)	NOT NULL	—	Enum: one_member_one_vote | stake_weighted | hybrid | role_based
governance_params	JSONB	NOT NULL	—	Thresholds, quorum, voting durations, etc.
charter	TEXT	NULL	NULL	Full charter text (Markdown)
member_count	INTEGER	NOT NULL	0	Denormalized
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
capital_pool_id	CHAR(36)	NULL	NULL	FK → capital_pool.id
status	VARCHAR(20)	NOT NULL	'active'	Enum: forming | active | suspended | dissolved
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: proposal ← Immutable after submission; outcomes appended

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK

Column	Type	Null	Default	Description
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id
proposer_id	CHAR(36)	NOT NULL	—	FK → account.id
proposal_type	VARCHAR(40)	NOT NULL	—	Enum: charter_amendment | capital_allocation | member_admission | member_removal | strategic | policy | general
title	VARCHAR(300)	NOT NULL	—	—
description	TEXT	NOT NULL	—	Full proposal in Markdown
status	VARCHAR(20)	NOT NULL	'open'	State machine — see Section 6.4
required_majority	DECIMAL(4,2)	NOT NULL	0.51	0.51 = simple majority, 0.67 = supermajority
quorum_pct	DECIMAL(4,2)	NOT NULL	0.10	Min fraction of eligible members who must vote
voting_opens_at	TIMESTAMPTZ	NOT NULL	—	—
voting_closes_at	TIMESTAMPTZ	NOT NULL	—	—
is_anonymous	BOOLEAN	NOT NULL	true	—
votes_for	INTEGER	NOT NULL	0	Denormalized counter

Column	Type	Null	Default	Description
votes_against	INTEGER	NOT NULL	0	—
votes_abstain	INTEGER	NOT NULL	0	—
eligible_voters	INTEGER	NOT NULL	0	Snapshotted at proposal open time
outcome	VARCHAR(20)	NULL	NULL	Enum: passed | failed | quorum_not_met | withdrawn — set on close
outcome_note	TEXT	NULL	NULL	—
ai_summary	TEXT	NULL	NULL	Generated within 2h of submission
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

4.11

AI Agent Schema

ai-agent-service database (kogi-ai-db)

Table: agent_session

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
user_id	CHAR(36)	NOT NULL	—	FK → account.id
session_type	VARCHAR(30)	NOT NULL	—	Enum: chat | task | synthesis | moderation | recommendation

Column	Type	Null	Default	Description
context	JSONB	NOT NULL	—	Serialized context: portfolio_id, wbs_id, active story list, etc.
messages	JSONB	NOT NULL	'[]'::jsonb	Conversation history for LLM context window
provider	VARCHAR(30)	NOT NULL	—	Enum: anthropic | openai | fallback
model	VARCHAR(60)	NOT NULL	—	e.g. claude-3-5-sonnet-20241022
tokens_used	INTEGER	NOT NULL	0	Cumulative tokens for billing
actions_taken	JSONB	NOT NULL	'[]'::jsonb	Autonomous actions taken with timestamps
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
expires_at	TIMESTAMPTZ	NOT NULL	—	Sessions expire after 2 hours of inactivity

Table: agent_action_log ← Audit trail of all autonomous AI actions

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
session_id	CHAR(36)	NULL	NULL	FK → agent_session.id, NULL for event-triggered

Column	Type	Null	Default	Description
user_id	CHAR(36)	NOT NULL	—	—
action_type	VARCHAR(60)	NOT NULL	—	e.g. create_story | send_notification | trigger_market_search
trigger_type	VARCHAR(30)	NOT NULL	—	Enum: user_request | event | schedule | automation_rule
trigger_ref	CHAR(36)	NULL	NULL	Event ID or rule ID that triggered the action
input_data	JSONB	NOT NULL	—	Data used by the agent to decide action
output_data	JSONB	NULL	NULL	Result of the action
status	VARCHAR(20)	NOT NULL	—	Enum: success | failed | overridden_by_user
reasoning	TEXT	NULL	NULL	Agent's reasoning summary for the action
override_at	TIMESTAMPTZ	NULL	NULL	If user overrode an agent action
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

✅ **Outbox Pattern — All Services**

- Every service DB includes: outbox_events(id, event_type, payload, status, created_at, published_at, retry_count)
- A per-service relay worker polls outbox every 100ms: SELECT ... WHERE status='PENDING' FOR UPDATE SKIP LOCKED LIMIT 50
- On NATS publish success: UPDATE outbox SET status='PUBLISHED', published_at=NOW()
- On NATS failure after 10 retries: status='DEAD_LETTER', alerting triggered
- This guarantees events are delivered even if the service crashes between DB commit and NATS publish

SECTION 5 · SERVICES

Service Layer Design

5.1 API Gateway (Go/Gin)

The API Gateway is the single entry point for all client traffic. It handles TLS termination, JWT validation, rate limiting, request routing, and response shaping. It does not contain business logic.

```
// api/server/server.go — core middleware chain

func (s *Server) buildRouter() *gin.Engine {
    r := gin.New()
    r.Use(
        middleware.RequestID(),        // inject X-Request-ID
        middleware.Logger(),           // structured log per request
        middleware.Recovery(),         // panic → 500, never crash
        middleware.RateLimit(s.redis), // token bucket per (IP + user_id)
        middleware.CORS(s.cfg),        // CORS headers
    )
    // Public routes (no auth)
    pub := r.Group("/api/v1")
    pub.GET("/health", s.healthHandler)
    pub.POST("/auth/register", s.registerHandler)
    pub.POST("/auth/login",    s.loginHandler)
    pub.GET("/market/listings",s.listingsHandler)  // unauthenticated browse

    // Authenticated routes
    auth := r.Group("/api/v1")
    auth.Use(middleware.JWT(s.cfg.JWTSecret))
    auth.Use(middleware.Idempotency(s.redis))  // check/store idempotency keys
    s.registerPortfolioRoutes(auth)
    s.registerWBSRoutes(auth)
    s.registerMarketRoutes(auth)
    s.registerExchangeRoutes(auth)
    s.registerCommunityRoutes(auth)
    s.registerWorkRoutes(auth)
    s.registerStudioRoutes(auth)
    s.registerAORoutes(auth)
    s.registerAIRoutes(auth)
    return r
}
```

JWT Token Structure

```
// JWT payload (claims)
```

```
{
  "sub":  "user-uuid",          // account ID
  "jti":  "session-uuid",       // session ID for revocation check
  "iat":  1700000000,            // issued at (unix)
  "exp":  1700086400,            // expires at = iat + 86400 (24h)
  "plan": "pro",                 // cached plan for feature gating
  "ws":   "workspace-uuid",      // active workspace
  "roles": ["user"],             // platform roles
  "ao":   ["ao-uuid-1", ...]     // AOs this user belongs to
}

// Validation steps in middleware:
// 1. Verify signature with HMAC-SHA256 secret
// 2. Check exp > now()
// 3. Check jti not in Redis revocation set (O(1) SET lookup)
// 4. Inject claims into gin.Context for downstream handlers
```

Rate Limiting — Token Bucket Algorithm

```
// Two-dimensional rate limit: per IP (unauthenticated) + per user (authenticated)
// Redis key: ratelimit:ip:{ip}  or  ratelimit:user:{user_id}
// Algorithm: Token Bucket with sliding window

type RateLimitConfig struct {
    BurstSize  int           // max tokens (burst capacity)
    RefillRate float64       // tokens per second
}

var limits = map[string]RateLimitConfig{
    "unauthenticated": {BurstSize: 60,    RefillRate: 1.0},   // 60/min
    "user_free":       {BurstSize: 600,   RefillRate: 10.0},  // 600/min
    "user_pro":        {BurstSize: 1000,  RefillRate: 16.7},  // 1000/min
    "user_enterprise": {BurstSize: 10000, RefillRate: 166.7}, // 10K/min
    "auth_endpoints":  {BurstSize: 10,    RefillRate: 0.167}, // 10 per min,
lockout after
}
```

5.2 Portfolio Service — Internal Design

Service Interface

```
// portfolio-svc/internal/service/portfolio.go

type PortfolioService interface {
    CreatePortfolio(ctx, req CreatePortfolioReq) (*Portfolio, error)
    GetPortfolio(ctx, id, viewerID string) (*Portfolio, error)
    UpdatePortfolio(ctx, id string, req UpdateReq, version int) (*Portfolio,
error)
```

```
    DeletePortfolio(ctx, id, actorID string) error
    ListPortfolios(ctx, ownerID string, opts ListOpts) ([]*Portfolio, int, error)

    CreateItem(ctx, portfolioID string, req CreateItemReq) (*PortfolioItem, error)
    GetItem(ctx, id, viewerID string) (*PortfolioItem, error)
    UpdateItem(ctx, id string, req UpdateItemReq, version int) (*PortfolioItem,
error)
    MoveItem(ctx, id, newParentID string) (*PortfolioItem, error)
    DeleteItem(ctx, id, actorID string) error

    ComputeHealthScore(ctx, portfolioID string) (int, error)
    GetPortfolioTree(ctx, portfolioID, viewerID string) (*PortfolioTree, error)
}

// All write operations:
// 1. Load entity + check version (optimistic lock)
// 2. Apply business rules (visibility, permissions)
// 3. Begin DB transaction
// 4. Write domain tables
// 5. Insert into outbox_events
// 6. Commit
// 7. Return updated entity
```

5.3 Exchange Service — Financial Correctness Patterns

```
// exchange-svc/internal/service/wallet.go
// ALL wallet operations use SELECT FOR UPDATE to prevent race conditions.

func (s *ExchangeService) FundEscrow(ctx context.Context, req FundEscrowReq) error
{
    return s.db.RunInTransaction(ctx, func(tx *sqlx.Tx) error {
        // 1. Lock wallet row — prevents concurrent balance changes
        wallet, err := s.repo.GetWalletForUpdate(tx, req.BuyerID)
        if err != nil { return err }

        // 2. Check sufficient available balance
        if wallet.FiatAvailable.LessThan(req.Amount) {
            return ErrInsufficientBalance
        }

        // 3. Debit available, credit reserved
        wallet.FiatAvailable = wallet.FiatAvailable.Sub(req.Amount)
        wallet.FiatReserved  = wallet.FiatReserved.Add(req.Amount)
        wallet.Version++
        if err := s.repo.UpdateWallet(tx, wallet); err != nil { return err }

        // 4. Write ledger entry (immutable)
        if err := s.repo.InsertLedgerEntry(tx, LedgerEntry{
            WalletID:     req.BuyerID,
            EntryType:    "hold",
            Amount:       req.Amount,
```

```
            BalanceAfter: wallet.FiatAvailable,
            ReferenceID:  req.EscrowID,
            ReferenceType:"escrow",
            Description:  fmt.Sprintf("Hold for escrow %s", req.EscrowID),
        }); err != nil { return err }

        // 5. Update escrow status + outbox event
        if err := s.repo.UpdateEscrowStatus(tx, req.EscrowID, "funded",
req.Version); err != nil { return err }
        return s.outbox.Insert(tx, "exchange.escrow.funded",
EscrowFundedPayload{...})
    })
}
```

5.4 Community Service — WebSocket Architecture

```
// community-svc/internal/ws/hub.go
// Hub manages all WebSocket connections for a room.
// Sharding: each room maps to a NATS subject; messages fan out via NATS across
pods.

type Hub struct {
    RoomID   string
    Clients  sync.Map         // map[clientID]*Client
    Inbound  chan *RoomMessage // from any client in this room
    nats     *nats.Conn
}

// Message flow for multi-pod deployment:
// 1. Client sends WS frame to pod A
// 2. Pod A validates, writes to DB, publishes to NATS subject "room.{roomID}"
// 3. All pods subscribed to "room.{roomID}" receive message
// 4. Each pod fans out to its local WebSocket clients in that room
// This ensures clients on different pods all receive the message.

func (h *Hub) Run() {
    sub, _ := h.nats.Subscribe("room."+h.RoomID, func(msg *nats.Msg) {
        h.Clients.Range(func(_, v any) bool {
            v.(*Client).Send <- msg.Data
            return true
        })
    })
    defer sub.Unsubscribe()
    // ... handle Inbound from local clients
}
```

5.5 AI Agent Service — LLM Integration

```go
// ai-agent-svc/internal/agent/agent.go

type AgentRequest struct {
    SessionID   string
    UserID      string
    TriggerType string  // "user_request" | "event" | "schedule" |
"automation_rule"
    UserMessage string
    Context     AgentContext  // portfolio, wbs, active stories, workspace
    PermittedActions []string // user-granted action types
}

func (a *Agent) Process(ctx context.Context, req AgentRequest) (*AgentResponse,
error) {
    // 1. Build system prompt with user context (portfolio summary, active
projects)
    sysPrompt := a.buildSystemPrompt(req.Context, req.PermittedActions)

    // 2. Load conversation history (last 20 turns for context window)
    history := a.sessionRepo.GetMessages(req.SessionID, 20)

    // 3. Call LLM with tool definitions
    resp, err := a.llm.Complete(ctx, CompletionRequest{
        Model:     a.cfg.Model,  // e.g. "claude-3-5-sonnet-20241022"
        System:    sysPrompt,
        Messages:  append(history, {Role:"user", Content:req.UserMessage}),
        MaxTokens: 4096,
        Tools:     a.getAvailableTools(req.PermittedActions),
    })

    // 4. Process tool calls (autonomous actions)
    actions := a.executeToolCalls(ctx, req.UserID, resp.ToolCalls)

    // 5. Log all actions to agent_action_log
    a.logActions(ctx, req.SessionID, req.UserID, actions)

    // 6. Persist session + updated message history
    a.sessionRepo.AppendMessage(req.SessionID, resp)

    return &AgentResponse{Text: resp.Text, Actions: actions}, nil
}
```

SECTION 6 · STATE MACHINES

State Machine Specifications

6.1

Story Status State Machine

All story types share this lifecycle; blocked is re-entrant

```
Valid status values and transitions:

  BACKLOG ──▶ READY ──▶ IN_PROGRESS ──▶ IN_REVIEW ──▶ DONE
      │          │          │              │
      └──────────┴──────────┴──▶ BLOCKED ──┘    (re-enters previous state on
unblock)
                                                 │
                  CANCELLED ◀────────────────────┘    (from any non-Done state)

BACKLOG:      Default state on creation. Not committed to any sprint.
READY:        Acceptance criteria defined; assigned executor confirmed.
IN_PROGRESS:  Executor actively working. start_date recorded.
BLOCKED:      Cannot proceed. Must have a blocker_note and linked blocker story or
resource.
IN_REVIEW:    Work submitted by executor, awaiting reviewer acceptance.
DONE:         Acceptance criteria verified. completed_at set. Immutable
thereafter.
CANCELLED:    Work abandoned. reason required.
```

State	On Entry	Valid Transitions	Guard Conditions
BACKLOG	Insert status_history entry; notify assignee if set	→ READY (acceptance criteria set + assigned)→ CANCELLED (anytime)	Guard: story_type must be valid; required fields present
READY	Notify assignee; add to sprint backlog if sprint assigned	→ IN_PROGRESS (executor starts work)→ BACKLOG (deprioritized)→ CANCELLED	Guard: assignee_id NOT NULL; story_points set (if agile WBS)
IN_PROGRESS	Record start timestamp; increment sprint WIP counter	→ IN_REVIEW (executor submits)→ BLOCKED (blocker identified)→ BACKLOG (de-scoped from sprint)→ CANCELLED	Guard: executor_id must match actor (or admin/AI override)
BLOCKED	Emit story.blocked event; AI Agent notified; auto-search Market	→ IN_PROGRESS (blocker resolved)→ CANCELLED	Guard: blocker_note NOT NULL; linked_blocker_id set

State	On Entry	Valid Transitions	Guard Conditions
IN_REVIEW	Notify reviewer(s); start review_deadline timer	→ DONE (approved)→ IN_PROGRESS (changes requested)→ BLOCKED	Guard: at least one reviewer assigned (or owner defaults)
DONE	Set completed_at; update sprint completed_points; emit story.completed; trigger OKR check-in	(Terminal — no transitions out)	Guard: acceptance_criteria NOT NULL; all sub-tasks Done; reviewer must be different from executor
CANCELLED	Emit story.cancelled; remove from sprint if assigned	(Terminal)	Guard: cancellation_reason NOT NULL

6.2

Escrow & Order State Machine

Order and Escrow states are synchronized

```
CREATED ──▶ FUNDED ──▶ DELIVERY_SUBMITTED ──▶ COMPLETE
                │              │
                │              └──▶ DISPUTED ──▶ RESOLVED
                └──▶ REFUNDED (if order cancelled before delivery)

Order status mirrors Escrow status with additional states:
  pending → accepted → in_progress → delivered → complete | disputed | cancelled
```

State	On Entry	Valid Transitions	Guard Conditions
CREATED	Generate escrow ID; notify buyer to fund	→ FUNDED (buyer deposits funds)→ CANCELLED (within 24h, no funds)	Guard: amount > 0; buyer wallet KYC status checked for amounts ≥ $1,000
FUNDED	Hold funds in reserved balance; notify seller to begin work; set review_deadline = now + configured_window	→ DELIVERY_SUBMITTED (seller submits delivery)→ REFUNDED (order cancelled before delivery, governance approval required)	Guard: wallet.FiatAvailable ≥ amount; fiat_reserved += amount; ledger entry written
DELIVERY_SUBMITTED	Start review countdown timer; notify buyer; AI Agent compares delivery to original brief	→ COMPLETE (buyer approves, or auto-release at review_deadline)→ DISPUTED (buyer opens dispute within window)→ FUNDED (seller resubmits — resets window)	Guard: delivery_notes NOT NULL; seller must be actor
DISPUTED	Freeze auto-release timer; create dispute record;	→ RESOLVED_FOR_SELLER	Guard: dispute_reason + evidence NOT NULL;

State	On Entry	Valid Transitions	Guard Conditions
	assign AI mediator (human escalation after 48h)	(release funds)→ RESOLVED_FOR_BUYER (full refund)→ RESOLVED_SPLIT (partial both)	opened within review_deadline
COMPLETE	Release reserved funds to seller wallet; deduct platform fee; write ledger entries; emit exchange.escrow.released; generate invoice artifact	(Terminal)	Guard: buyer_approved=true OR auto_release_at ≤ NOW(); platform_fee calculated at fee_pct
REFUNDED	Return reserved funds to buyer available balance; write compensating ledger entries; emit exchange.escrow.refunded	(Terminal)	Guard: funds fully in reserved (not yet released); governance approval if required by AO rules

6.3

Crowdfunding Campaign State Machine

```
DRAFT ──► APPROVED ──► ACTIVE ──► FUNDED ──► DELIVERING ──► COMPLETE
                          │
                          └──► FAILED (end_date reached, goal not met)
                          └──► CANCELLED (owner withdraws)
```

State	On Entry	Valid Transitions	Guard Conditions
DRAFT	Owner building campaign; not visible publicly	→ APPROVED (platform review passes, or auto-approve if owner verified)	Guard: title, description, goal_amount, end_date, at least 1 tier all set
APPROVED	Campaign visible but not yet accepting contributions; launch date set	→ ACTIVE (start_date reached or owner manually launches)	Guard: legal review complete for equity campaigns; KYC verified for equity/revenue_share
ACTIVE	Accepting contributions; progress bar displayed; notifications at 25/50/75/100%	→ FUNDED (raised_amount ≥ goal_amount, all-or-nothing)→ FAILED (end_date passed, raised < goal)→ CANCELLED (owner withdraws)	Scheduler checks hourly; partial fund campaigns can reach DELIVERING before 100%
FAILED	Emit campaign.failed; trigger full refund of all contributions via Exchange	→ (Terminal for all-or-nothing; keep-what-you-raise campaigns skip this state)	Guard: end_date < NOW() AND raised_amount <

State	On Entry	Valid Transitions	Guard Conditions
			goal_amount AND model = all_or_nothing
FUNDED	Funds released from campaign escrow to owner Capital Pool; emit campaign.funded; cap table updated for equity campaigns	→ DELIVERING (owner begins fulfillment)	Guard: raised_amount ≥ goal_amount
DELIVERING	Owner posting updates; backer rewards being fulfilled; equity stakeholders recorded	→ COMPLETE (owner marks deliveries complete)	—
COMPLETE	Campaign closed; all rewards delivered; final report generated by AI Agent	(Terminal)	—

6.4

AO Proposal State Machine

State	On Entry	Valid Transitions	Guard Conditions
OPEN	Emit proposal.submitted; trigger AI summary generation; notify all eligible voters; snapshot eligible_voter_count	→ CLOSED (voting_closes_at reached)→ WITHDRAWN (proposer withdraws before close)	Guard: ao_id valid; proposer is AO member with proposal rights; voting window ≥ 24h
CLOSED	Tally: votes_for / eligible_voters ≥ quorum_pct → compute result; outcome = passed if votes_for / (votes_for + votes_against) ≥ required_majority	→ PASSED (majority + quorum met)→ FAILED (quorum met, majority not)→ QUORUM_NOT_MET (quorum not reached)	Scheduler fires at voting_closes_at; auto-executed
PASSED	Emit proposal.passed; trigger downstream effects (capital transfer, charter update, member admission/removal)	(Terminal for record; downstream actions are new events)	Guard: downstream services receive events and execute changes within 1h SLA
FAILED	Emit proposal.failed; no downstream action; record in governance ledger	(Terminal)	—

State	On Entry	Valid Transitions	Guard Conditions
WITHDRAWN	Proposer must be actor; no refund if contribution was made	(Terminal)	Guard: status=OPEN at time of withdrawal

6.5

Idea Studio Pipeline State Machine

State	On Entry	Valid Transitions	Guard Conditions
SPARK	Auto-process capture modality (transcribe, summarize, describe); create AI-generated title if blank; add to inbox queue; emit studio.idea.captured	→ CONCEPT (user advances)→ PARKED (user defers)	Guard: raw_capture NOT NULL; title generated if blank; capture_modality set
CONCEPT	Trigger AI synthesis pass: problem reframing, viability score, competitor scan, audience identification, 10 questions; emit studio.synthesis.complete	→ RESEARCH (user advances to validation)→ SPARK (user reverts to rework capture)→ PARKED	Guard: AI synthesis must complete before advance is allowed (async check); synthesis retried up to 3x on LLM failure
RESEARCH	Make framework templates available (BMC, SWOT, etc.); AI populates first draft of selected frameworks; emit studio.research.started	→ PROTOTYPE (user advances)→ CONCEPT (back if hypothesis changed)→ PARKED	Guard: problem_statement NOT NULL required before advance; at least 1 framework partially completed
PROTOTYPE	Enable Prototype story creation; track build iterations; AI generates design brief after first prototype iteration	→ DESIGN (design brief generated)→ RESEARCH (back)→ PROMOTED (skip design, fast-track to portfolio)	Guard: at least 1 prototype story must exist and be in IN_PROGRESS or DONE status
DESIGN	Design critique by AI Agent; IP timestamp offered; presentation builder available	→ PROMOTED (user promotes to Portfolio project)→ PROTOTYPE (back for revision)→ PARKED	Guard: design_brief NOT NULL
PARKED	Idea preserved but removed from active inbox view; can be restored	→ any previous stage (user unparks)	—
PROMOTED	Create Portfolio Project + WBS from idea; link back to idea record; emit studio.idea.promoted	(Terminal for idea record; portfolio item is the active artifact)	Guard: target portfolio_id specified; user has write access to portfolio

SECTION 7 · ALGORITHMS

Algorithm Specifications

7.1 Market Recommendation Engine

The recommendation engine combines three signals to rank listings for a given user: Portfolio Context Match (what the user currently needs based on active stories and resource gaps), Collaborative Filtering (what similar users engaged with), and Quality Score (reputation, freshness, response rate).

Algorithm: Market Recommendation Ranking

Input: user_id: string, limit: int, context: {active_wbs_ids, blocked_story_ids, portfolio_tags}

Output: Ranked list of listing_ids with scores

1. Compute Portfolio Signal: for each active WBS story in context, extract skill requirements and missing resource types. Build skill_vector[N] from platform skill taxonomy.
2. Retrieve Candidate Set: Elasticsearch semantic query using skill_vector + portfolio_tags → top 200 listings (recall pool).
3. Compute Content Score for each candidate: cosine_similarity(listing_embedding, user_skill_vector) → [0,1].
4. Compute Collaborative Score: SELECT avg(engagement_weight) FROM listing_events WHERE listing_id IN candidates AND user_id IN (SELECT similar_user FROM user_similarity WHERE user_id=? AND similarity > 0.7) → [0,1].
5. Compute Quality Score: (0.4 × avg_rating/5) + (0.3 × response_rate) + (0.2 × completion_rate) + (0.1 × recency_decay) → [0,1].
6. Compute Blocked Story Boost: if listing.tags ∩ blocked_story.required_skills ≠ ∅, multiply final score × 1.5.
7. Final Score = 0.45×content + 0.25×collaborative + 0.25×quality + 0.05×diversity_penalty.
8. Return top(limit) by final score; exclude already-ordered or dismissed listings.

Complexity: O(N log N) where N = candidate pool size (~200). Elasticsearch query O(log M) M = total listings.

7.2 Reputation Scoring

Reputation is a composite score used for search ranking, governance eligibility, and trust display. It is recomputed nightly for all active users.

Algorithm: User Reputation Score

Input: user_id: string, lookback_days: int (default 365)

Output: reputation_score: float [0–100]

9. Review Component (40%): avg_rating over lookback period, weighted by recency (reviews in last 30d weight 2×). Normalized to [0,1]. Score = weighted_avg / 5.0.

10. Delivery Component (25%): completed_orders / total_accepted_orders over lookback. Include only orders where user was seller.
11. Response Component (15%): avg(response_time_hours < 24h ? 1 : max(0, 1 - (response_time_hours - 24)/72)) across all orders in lookback.
12. Community Component (10%): min(1.0, (space_contributions + story_completions_in_AOs + governance_votes_cast) / 50) — community activity normalized.
13. Verification Bonus (10%): email_verified (2pts) + ID_verified (4pts) + professional_license_verified (4pts). Score = total_pts / 10.
14. Final = 100 × (0.40×review + 0.25×delivery + 0.15×response + 0.10×community + 0.10×verification).
15. Apply penalty: if fraud_flags > 0 in last 90d, multiply final × (1 - 0.2 × min(fraud_flags, 3)).
16. Clamp to [0, 100] and round to 1 decimal place. Write to profile.reputation_score.

Complexity: O(1) with pre-aggregated review/order stats. Full recompute runs nightly via batch job.

7.3 OKR Auto-Scoring

Algorithm: OKR Period-End Auto-Scorer

Input: okr_id: string, period_end_date: date

Output: okr.score: float [0.0–1.0] and updated status

17. For each KeyResult KR in okr.key_results:
18. a. Pull all linked story completions from WBS service since period_start.
19. b. If KR has numeric unit: KR.score = min(1.0, KR.current_value / KR.target_value).
20. c. If KR is binary (done/not done): KR.score = KR.current_value > 0 ? 1.0 : 0.0.
21. d. If KR links to a sub-OKR: KR.score = sub_okr.score (recursive).
22. Compute weighted average: okr.score = Σ(KR.score × KR.weight) / Σ(KR.weight). Default KR weight = 1.0.
23. Derive status: score ≥ 0.7 → "complete", 0.4–0.69 → "off_track", < 0.4 → "missed". Mark period as closed.
24. Emit work.okr.scored event with score, status, per-KR breakdown.
25. If parent_okr_id set: trigger parent OKR re-score to propagate up the cascade.

Complexity: O(K × S) where K = key results, S = linked stories. Runs at period_end_date + 1 day via scheduler.

7.4 Sprint Planning Optimizer

Algorithm: AI-Assisted Sprint Planner

Input: wbs_id: string, sprint_duration_days: int, team_capacity_points: int

Output: Recommended story_ids[] with rationale

26. Fetch all backlog stories for wbs_id sorted by: (priority_rank × 0.6) + (stakeholder_request_score × 0.2) + (age_in_backlog_days/30 × 0.2). Priority rank: critical=4, high=3, medium=2, low=1.
27. Fetch last 6 completed sprints velocity data. Compute adjusted_capacity = median(velocity_6) × team_capacity_points (handle first sprint: use 0.7 × team_capacity as default).
28. Remove stories with unresolved blockers (status=BLOCKED with no resolution estimate).

29. Remove stories with dependencies on stories not yet in DONE status (topological ordering).
30. Greedy bin-pack: iterate priority-sorted backlog, add story to sprint if running_points + story.story_points ≤ adjusted_capacity.
31. Apply diversity constraint: no more than 60% of sprint points from a single story_type to avoid monoculture sprints.
32. Compute confidence score: 1.0 - abs(packed_points - adjusted_capacity) / adjusted_capacity.
33. Return packed story list with: total_points, confidence, adjusted_capacity, and per-story rationale.

Complexity: O(N log N) for sort + O(N) for greedy pack. N = backlog size.

7.5 AI Agent Tool Routing

Algorithm: Agent Action Router

Input: user_message: string, context: AgentContext, permitted_actions: []string

Output: ActionPlan{tool_calls[], response_strategy}

34. Parse intent from user_message using LLM classifier: categories = [query, create, update, search, analyze, automate, other].
35. If intent=query: route to read-only tool (get_portfolio, search_market, get_story_status, etc.). No state mutation.
36. If intent=create/update: verify action_type is in permitted_actions. If not permitted, respond with explanation and request permission grant.
37. Build tool call list: for each identified action, select the lowest-privilege tool that satisfies the intent.
38. Check rate limits: max 10 autonomous tool calls per user per minute; max 50 per hour. Soft-fail (inform user) rather than error.
39. Execute tools sequentially for dependent actions (create portfolio → create item in portfolio); parallel for independent actions.
40. For each tool call result: if error, attempt one retry with corrected parameters; on second failure, surface error to user.
41. Compile final response: executed_actions[] + text_response with reasoning summary.

Complexity: O(T) where T = number of tool calls. LLM latency dominates.

7.6 Portfolio Health Score

Algorithm: Portfolio Health Scorer

Input: portfolio_id: string

Output: health_score: int [0–100]

42. Fetch all portfolio items (excluding deleted). Compute status distribution: active_pct, on_hold_pct, cancelled_pct, overdue_pct.
43. Item Health (30%): score = active_pct × 1.0 + planning_pct × 0.7 - overdue_pct × 2.0 - cancelled_pct × 0.5. Clamp [0,1].
44. Resource Utilization (20%): for Capital resources, utilization = used_budget / total_budget. For Labor, utilization = hours_logged / capacity. Average across resource types. Score = min(1, utilization × 1.2) (slight bonus for near-full utilization).

45. Goal Progress (30%): fetch linked OKRs. Average OKR scores across all linked OKRs. If no OKRs, use story completion rate for active projects.
46. Activity Recency (10%): decay_factor = e^(-days_since_last_update / 30). Score = decay_factor.
47. Risk Penalty (10%): for each open Risk story, apply penalty: critical=-0.15, high=-0.08, medium=-0.04. Aggregate, clamp to [-0.5, 0].
48. Final = 100 × clamp(0,1, 0.30×item + 0.20×resource + 0.30×goal + 0.10×recency + 0.10×risk).

Complexity: O(P + R + O) P=portfolio items, R=resources, O=linked OKRs. Scheduled daily + on significant events.

SECTION 8 · EVENTS

Event Catalog — Full Payload Schemas

All events are JSON-serialized and wrapped in an EventEnvelope (see Section 3.2). The payloads below describe the data field of the envelope.

8.1 User & Auth Events

Event Type	Source	Payload Fields	Consumers
user.registered	auth-svc	user_id, email, plan, created_at	Provisioner, Analytics
user.provisioned	kernel	user_id, workspace_id, portfolio_id, wallet_id	Community (profile init), Market, Email
user.email_verified	auth-svc	user_id, email, verified_at	Analytics, Email (welcome)
user.profile_updated	auth-svc	user_id, updated_fields[], version	Community (feed), Market (listing update), Search
user.plan_changed	auth-svc	user_id, old_plan, new_plan, effective_at	All services (feature gates), Analytics
user.session_revoked	auth-svc	user_id, session_id, reason	API Gateway (cache invalidate)

8.2 Portfolio Events

Event Type	Source	Payload Fields	Consumers
portfolio.created	portfolio-svc	portfolio_id, owner_id, owner_type, name	Analytics, Community (if public)
portfolio.item.created	portfolio-svc	item_id, portfolio_id, item_type, name, parent_id?	WBS (if project), Analytics, AI Agent
portfolio.item.status_changed	portfolio-svc	item_id, from_status, to_status, actor_id	Work (OKR check), Analytics, Community
portfolio.health_score_updated	portfolio-svc	portfolio_id, score, prev_score, factors	AI Agent (alert if drop > 10)
portfolio.resource.updated	portfolio-svc	resource_id, resource_type, field_changes, version	Analytics, AI Agent

8.3 WBS & Story Events

Event Type	Source	Payload Fields	Consumers
story.created	wbs-svc	story_id, wbs_id, story_type, title, priority, created_by	Analytics, AI Agent (classify + suggest)
story.status_changed	wbs-svc	story_id, from_status, to_status, actor_id, actor_type, sprint_id?	Work (OKR update), Exchange (milestone), Analytics
story.blocked	wbs-svc	story_id, blocker_note, blocker_story_id?, duration_hours	AI Agent (escalate + market search), Community (notify)
story.completed	wbs-svc	story_id, wbs_id, story_type, story_points, executor_id	Sprint (velocity), OKR (check-in), Exchange (milestone release), AI (contributor reward)
sprint.started	wbs-svc	sprint_id, wbs_id, story_ids[], planned_points, start_date	Analytics, AI (capacity check), Notifications
sprint.closed	wbs-svc	sprint_id, velocity, completed_pct, retro_story_id	Analytics, Work (OKR check-in trigger), AI (generate retro)

8.4 Market Events

Event Type	Source	Payload Fields	Consumers
market.listing.created	market-svc	listing_id, provider_id, listing_type, category, title	Search (index), Community (feed), AI (recommend)
market.order.placed	market-svc	order_id, listing_id, buyer_id, seller_id, amount, currency	Exchange (create escrow), Portfolio (create item), WBS (create story), Notifications
market.order.delivered	market-svc	order_id, delivery_notes, review_deadline	Exchange (start auto-release timer), AI (compare to brief), Notifications
market.order.completed	market-svc	order_id, amount, portfolio_item_id, wbs_story_id	Exchange (release escrow), Review (prompt), Analytics
market.order.disputed	market-svc	order_id, reason, evidence_urls[], opened_by	Exchange (freeze escrow), Dispute service, Notifications
market.review.submitted	market-svc	review_id, order_id, reviewer_id, reviewee_id, rating, dims	Reputation engine (recalculate), Listing (update avg_rating)

8.5 Exchange Events

Event Type	Source	Payload Fields	Consumers
exchange.escrow.created	exchange-svc	escrow_id, buyer_id, seller_id, amount, reference_id, reference_type	Market (update order status), Notifications
exchange.escrow.funded	exchange-svc	escrow_id, amount, funded_at	Market (order: accepted), Notifications
exchange.escrow.released	exchange-svc	escrow_id, to_party_id, net_amount, fee, released_at	Portfolio (capital update), Tax svc, Notifications
exchange.escrow.refunded	exchange-svc	escrow_id, to_party_id, amount, reason	Portfolio (capital update), Tax svc, Notifications
exchange.distribution.completed	exchange-svc	distribution_id, ao_id, total_amount, recipients[{user_id, amount}]	AO ledger, Portfolio (capital), Tax, Notifications
exchange.credit.awarded	exchange-svc	user_id, credits, reason, source_type, source_id	Community (badge?), Analytics

8.6 AI Agent Events

Event Type	Source	Payload Fields	Consumers
ai.synthesis.completed	ai-svc	session_id, idea_id?, story_id?, synthesis_type, summary, viability_score?	Studio (update idea), WBS (update story), Notifications
ai.action.taken	ai-svc	session_id, user_id, action_type, action_params, result, reasoning	Audit log, Analytics, User notifications if impactful
ai.alert.generated	ai-svc	user_id, alert_type, context_ref, message, urgency	Notifications (in-app + email if urgency=high)
ai.recommendation.ready	ai-svc	user_id, recommendation_type, items[], generated_at	API cache invalidate, Notifications

SECTION 9 · API DESIGN

API Design — Request/Response Contracts

API Design Conventions

- Base URL: https://api.kogi.io/api/v1/
- Authentication: Bearer {JWT} in Authorization header for all protected endpoints
- Idempotency: POST/PUT/PATCH accept Idempotency-Key header (UUID); duplicates return 200 with cached response
- Pagination: cursor-based for lists. Response includes {data[], next_cursor?, prev_cursor?, total_count?}
- Errors: always {error: {code, message, details?, request_id}}
- Versioning: /api/v1/... — breaking changes bump major version with 12-month notice
- Content-Type: application/json for all request and response bodies
- Timestamps: ISO 8601 with UTC timezone (2024-01-15T10:30:00Z)

9.1 Standard Response Envelope

```
// Success response
{
  "data":     { /* resource or array */ },
  "meta":     { "request_id": "uuid", "timestamp": "ISO8601" },
  "pagination": { "next_cursor": "opaque-string", "total_count": 142 }  // if list
}

// Error response
{
  "error": {
    "code":       "RESOURCE_NOT_FOUND",   // machine-readable code (see Section
12)
    "message":    "Portfolio not found",   // human-readable
    "details":    { "field": "id", "value": "xyz" },  // optional
    "request_id": "uuid"
  }
}
```

9.2 Portfolio API — Key Endpoints

```
POST /portfolios
Request:
  { "name": string, "description"?: string, "visibility":
"private"|"team"|"org"|"public",
```

```
    "tags"?: string[], "metadata"?: object }
Response 201:
  { "data": Portfolio }
Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED, 429 RATE_LIMITED

GET /portfolios/:id
Query params: ?include=items,resources,health_score
Response 200:
  { "data": Portfolio & { items?: PortfolioItem[], health_score?: int } }
Errors: 403 FORBIDDEN (visibility check), 404 NOT_FOUND

PUT /portfolios/:id/items
Request:
  { "item_type": "program"|"project"|"sub_portfolio"|"resource"|"container",
    "name": string, "description"?: string, "parent_id"?: string,
    "metadata": object, "visibility"?: string }
Header: Idempotency-Key: {uuid}
Response 201:
  { "data": PortfolioItem }
Errors: 400, 403, 404, 409 CONFLICT (optimistic lock failure)
```

9.3 WBS / Story API — Key Endpoints

```
POST /wbs/:id/stories
Request:
  { "story_type": string, "title": string, "description"?: string,
    "priority": "critical"|"high"|"medium"|"low",
    "parent_id"?: string, "story_points"?: int,
    "assignee_id"?: string, "executor_type"?: string,
    "due_date"?: string, "labels"?: string[] }
Response 201:
  { "data": Story }

PUT /stories/:id/status
Request:
  { "status": string, "note"?: string, "version": int }
  // version required for optimistic locking
Response 200:
  { "data": Story }
Errors: 409 VERSION_CONFLICT (story was updated concurrently)

POST /wbs/:id/sprints/:sprint_id/plan
Request:
  { "use_ai_suggestion": boolean,
    "story_ids"?: string[],    // override if use_ai_suggestion=false
    "capacity_points": int }
Response 200:
  { "data": { "sprint": Sprint, "stories": Story[], "ai_confidence"?: float } }
```

9.4 Exchange API — Critical Financial Endpoints

```
POST /exchange/escrow
Header: Idempotency-Key: {uuid}    // REQUIRED — prevents double-escrow
Request:
  { "reference_id": string,         // market order ID
    "reference_type": "market_order"|"contract"|"campaign",
    "amount": number,               // in USD, positive
    "buyer_id": string,
    "seller_id": string,
    "conditions"?: ConditionArray,
    "milestones"?: MilestoneArray }
Response 201:
  { "data": Escrow }
Errors: 400 INVALID_AMOUNT, 402 INSUFFICIENT_BALANCE, 403 KYC_REQUIRED

PUT /exchange/escrow/:id/release
Header: Idempotency-Key: {uuid}    // REQUIRED
Request:
  { "approver_id": string, "version": int }
Response 200:
  { "data": { "escrow": Escrow, "net_amount": number, "fee": number } }

// Financial endpoint rate limit: 30 requests/minute (stricter than default)
```

9.5 AI Agent API — Streaming Endpoint

```
POST /ai/agent/chat
Request:
  { "session_id"?: string,    // null for new session
    "message": string,
    "context"?: {              // optional overrides
      "portfolio_id"?: string,
      "wbs_id"?: string,
      "active_story_ids"?: string[]
    },
    "permitted_actions"?: string[]  // pre-authorized action types
  }
Response: Server-Sent Events stream
  Content-Type: text/event-stream
  Events:
    data: {"type":"text_delta", "delta":"...partial text..."}
    data: {"type":"tool_call", "tool":"create_story", "params":{...}}
    data: {"type":"tool_result", "tool":"create_story", "result":{...}}
    data: {"type":"complete", "session_id":"...", "actions_taken":[...]}

GET /ai/agent/sessions/:id
Response: full conversation history + actions_taken[]
```

SECTION 10 · CACHING

Caching Architecture

The platform uses a three-level caching strategy. Redis Cluster serves as the primary cache (L1). PostgreSQL read replicas with materialized views serve as L2. For read-heavy aggregate queries, a ClickHouse OLAP store serves as L3.

10.1 Redis Cache Key Schema

Cache Key Pattern	TTL	Value	Invalidated By
user:{id}:profile	15m	JSON Profile object	user.profile_updated event
user:{id}:session:{jti}:valid	24h	1 (exists = valid)	user.session_revoked event
portfolio:{id}	5m	JSON Portfolio object	portfolio.* events for this id
portfolio:{id}:tree	2m	JSON tree of all items	portfolio.item.* events for this portfolio
portfolio:{id}:health	1h	int health score	portfolio.health_score_updated
wbs:{id}:board	60s	JSON board state (all columns)	story.status_changed for this WBS
wbs:{id}:sprint_active	2m	JSON current sprint + stories	sprint.* events for this WBS
market:listings:search:{hash}	2m	JSON search results page	market.listing.* events (TTL only, no targeted invalidation for searches)
market:listing:{id}	5m	JSON Listing object	market.listing.* events for this id
market:recommendations:{uid}	10m	JSON list of listing_ids	user.profile_updated, portfolio.item.created, story.blocked
exchange:wallet:{uid}:balances	10s	JSON balance object	exchange.escrow.* events for this user — short TTL for financial accuracy
community:space:{id}:meta	10m	JSON Space metadata	space.* events for this id
community:room:{id}:recent	30s	JSON last 50 messages	message.created in this room

Cache Key Pattern	TTL	Value	Invalidated By
ai:recommendations:{uid}	30m	JSON recommendations payload	ai.recommendation.ready for this user
ratelimit:ip:{ip}	60s	Token count (float)	TTL-based only
ratelimit:user:{uid}	60s	Token count (float)	TTL-based only
idempotency:{key}	24h	JSON cached response	TTL-based only

10.2 Cache Invalidation Patterns

```
// Pattern 1: Direct key invalidation (single resource)
// Event: portfolio.item.created { item_id, portfolio_id }
func handlePortfolioItemCreated(event Event) {
    redis.Del(ctx, "portfolio:"+event.PortfolioID)
    redis.Del(ctx, "portfolio:"+event.PortfolioID+":tree")
    // Do NOT invalidate all search caches — too expensive; let them expire
}

// Pattern 2: Tag-based invalidation (group of related keys)
// Example: when an AO's membership changes, invalidate all Space caches for that
AO
// Redis SMEMBERS ao:{ao_id}:cache_keys → DEL each key

// Pattern 3: Version-tagged cache (for optimistic concurrency)
// Key: portfolio:{id}:v:{version}
// On read: GET portfolio:{id}:version → compare with DB version
// On mismatch: DEL old key, re-fetch from DB, SET new key

// Pattern 4: Write-through (for high-read, low-write resources)
// On DB write: immediately SET the new value in Redis before returning response
// Used for: user profile, listing details, wallet balances
```

10.3 Write-Through vs Read-Through Decision

Resource	Strategy	Reason
User Profile	Write-through	High read, low write; staleness is costly (auth, display)
Wallet Balance	Write-through, 10s TTL	Financial accuracy; short TTL as safety net
Listing Details	Write-through	High read (search results), moderate write
WBS Board State	Read-through, 60s TTL	Frequent small writes; acceptable slight staleness

Resource	Strategy	Reason
Sprint Active	Read-through, 2m TTL	Moderate read; can tolerate slight lag
Search Results	Read-through, 2m TTL	Too broad for targeted invalidation; TTL sufficient
Recommendations	Write-through on event	Point-in-time freshness needed when context changes
Governance Proposals	No cache	Voting correctness requires DB reads always
Ledger Entries	No cache	Financial immutability — always read from DB

SECTION 11 · SECURITY

Security Design

11.1 Encryption

Data at Rest	AES-256-GCM. RDS storage encryption with AWS KMS. S3 SSE-S3. Redis ElastiCache encryption at rest.
Data in Transit	TLS 1.3 minimum (TLS 1.2 fallback for legacy clients). HSTS with max-age 31536000. Certificate pinning on mobile.
PII & Financial Data	Field-level encryption via AWS KMS for: password_hash (Argon2id), mfa_secret, SSN/tax_id (if collected), bank account numbers, card data (tokenized via Stripe — never touches Kogi DB).
JWT Signing	HMAC-SHA256 with 256-bit secret rotated every 90 days. Rotation uses overlap window: old key valid for 24h post-rotation to allow session expiry.
API Keys	Generated as: HMAC-SHA256(secret, user_id + timestamp + random_bytes). Stored as SHA-256 hash; full key shown once at creation only.

11.2 Input Validation & Injection Prevention

```
// ALL SQL uses parameterized queries via sqlx — no string interpolation.
// Example of correct pattern:
query := "SELECT * FROM story WHERE id = $1 AND wbs_id = $2 AND deleted_at IS
NULL"
row := db.QueryRowContext(ctx, query, storyID, wbsID)

// All user input sanitized at handler layer before reaching service:
type CreateStoryReq struct {
    Title       string `json:"title"       validate:"required,min=1,max=500"`
    Description string `json:"description" validate:"omitempty,max=50000"`
    Priority    string `json:"priority"    validate:"oneof=critical high medium
low"`
    StoryType   string `json:"story_type"  validate:"required,story_type"`  //
custom validator
    StoryPoints *int   `json:"story_points" validate:"omitempty,min=0,max=999"`
}

// HTML output: all user-provided content rendered as Markdown and sanitized
// through allowlist-based HTML sanitizer before serving to clients.
// allowlisted tags:
<p><br><strong><em><code><pre><ul><ol><li><h1-h4><a><blockquote>
```

11.3 Authorization — Permission Model

```
// Three-layer authorization: Platform → Workspace → Resource

// Layer 1: Platform role (from JWT)
// roles: user | moderator | platform_admin | financial_admin

// Layer 2: Workspace role (loaded from DB, cached 5min)
// roles: owner | admin | member | guest

// Layer 3: Resource-level ACL (checked per request)
// visibility: private | team | org | public
// collaborators: [{user_id, role}] for direct grants

// Authorization decision pseudocode:
func CanAccess(actor Actor, resource Resource, action Action) bool {
    if actor.IsPlatformAdmin() { return true }           // superuser
    if resource.OwnerID == actor.UserID { return true }  // owner
    if action == READ && resource.Visibility == PUBLIC { return true }
    wsRole := GetWorkspaceRole(actor.UserID, resource.WorkspaceID)
    if wsRole == OWNER || wsRole == ADMIN { return true }
    if wsRole == MEMBER && action == READ && resource.Visibility == TEAM { return
true }
    for _, collab := range resource.Collaborators {
        if collab.UserID == actor.UserID {
            return RolePermits(collab.Role, action)
        }
    }
    return false  // deny by default
}
```

11.4 Secret Management

Secret	Storage	Access Pattern	Rotation
DB passwords	AWS Secrets Manager	Lambda rotation; injected at pod startup via AWS SDK	Every 30 days — automatic
JWT signing secret	AWS Secrets Manager	Read at startup + cached in memory	Every 90 days — 24h overlap
LLM API keys	AWS Secrets Manager	Read per service startup, never logged	On provider rotation schedule
Stripe API keys	AWS Secrets Manager	Read per service startup	On breach or quarterly
Redis password	AWS Secrets Manager	Injected via K8s Secret (Secrets Store CSI)	Every 90 days
Webhook signing secrets	AWS Secrets Manager	Per-webhook, read on webhook creation and verification	On customer request

Secret	Storage	Access Pattern	Rotation
Internal service tokens	K8s ServiceAccount + Istio mTLS	Istio SPIFFE cert auto-rotated	Every 24h (SPIFFE cert)

SECTION 12 · ERRORS

Error Handling & Taxonomy

12.1 Error Code Taxonomy

Code	HTTP Status	Category	Retry?	Description
VALIDATION_ERROR	400	Client	No	Request body fails validation (missing required field, invalid enum, etc.)
UNAUTHORIZED	401	Client	No	Missing or invalid JWT token
TOKEN_EXPIRED	401	Client	Yes	JWT expired; client should refresh token and retry
FORBIDDEN	403	Client	No	Authenticated but lacks permission for this resource or action
KYC_REQUIRED	403	Client	No	KYC verification required before this financial operation
NOT_FOUND	404	Client	No	Resource not found or soft-deleted
CONFLICT	409	Client	Yes	Optimistic lock failure (version mismatch); client must re-fetch and retry
IDEMPOTENT_REPLAY	200	Client	N/A	Duplicate idempotency key — returns cached original response
INSUFFICIENT_BALANCE	402	Client	No	Wallet has insufficient available balance for this transaction
RATE_LIMITED	429	Client	Yes	Rate limit exceeded; Retry-After header included
STORY_INVALID_TRANSITION	422	Business	No	Story status transition not permitted by state machine
GOVERNANCE_REQUIRED	403	Business	No	Action requires AO governance vote; proposal must be submitted first
ESCROW_AUTO_RELEASED	409	Business	No	Dispute submitted after auto-release deadline
SPRINT_CAPACITY_EXCEEDED	422	Business	No	Adding story would exceed sprint capacity; reject or override
INTERNAL_ERROR	500	Server	Yes	Unexpected server error; request_id included for support
SERVICE_UNAVAILABLE	503	Server	Yes	Downstream service unreachable; Retry-After header included
DATABASE_ERROR	500	Server	Yes	Database write/read failure (logged with request_id)
EVENT_BUS_ERROR	500	Server	Yes	NATS publish failure (transaction rolled back; idempotent retry safe)

Code	HTTP Status	Category	Retry?	Description
AI_PROVIDER_ERROR	503	External	Yes	LLM provider returned error; fallback to secondary provider if available
PAYMENT_PROVIDER_ERROR	502	External	Yes	Stripe returned error; payment not processed; safe to retry

12.2 Error Handling Patterns

```
// Pattern 1: Structured error wrapping (Go)
type KogiError struct {
    Code      ErrorCode
    Message   string
    Details   map[string]interface{}
    Cause     error  // wrapped original error (not exposed to clients)
}

// All service functions return (result, *KogiError)
// HTTP handler maps KogiError.Code to HTTP status

// Pattern 2: Retry policy for transient errors
type RetryPolicy struct {
    MaxAttempts   int
    BaseDelay     time.Duration
    MaxDelay      time.Duration
    Multiplier    float64
    Jitter        bool
    RetryableErrors []ErrorCode  // e.g. INTERNAL_ERROR, SERVICE_UNAVAILABLE
}

DefaultRetryPolicy = RetryPolicy{
    MaxAttempts: 3, BaseDelay: 100ms, MaxDelay: 5s, Multiplier: 2.0, Jitter: true
}

// Pattern 3: Circuit breaker (per downstream service)
// States: CLOSED (normal) → OPEN (failing) → HALF_OPEN (testing)
// Threshold: 5 failures in 10s → OPEN for 30s → HALF_OPEN (1 probe)
// On OPEN: return SERVICE_UNAVAILABLE immediately without calling downstream
```

12.3 Distributed Transaction Error Recovery

```
// Saga pattern for multi-service operations (e.g. market order placement)

// Forward steps:                  Compensating steps (on failure):
// 1. market-svc: create order   → delete order
// 2. portfolio-svc: create item → delete item
// 3. wbs-svc: create story      → delete story
```

```
// 4. exchange-svc: create escrow → (if funded) refund escrow
// 5. exchange-svc: fund escrow

// If step 3 fails: execute compensations for steps 2, 1 in reverse order.
// Compensation events published to NATS; each service handles its own rollback.
// Saga coordinator (in market-svc) tracks step status in saga_state table.

// saga_state table schema:
// id, saga_type, reference_id, current_step, status, steps_completed[],
// steps_compensated[], failure_reason, created_at, updated_at

// Idempotency ensures compensations are safe to replay if network fails mid-saga.
```

SECTION 13 · BACKGROUND JOBS

Background Job Design

13.1 Job Registry

Job Name	Schedule	Service	Description	Timeout	Idempotent?
outbox-relay	continuous (100ms poll)	Every service	Publish pending outbox events to NATS	5s	Yes
session-expiry-sweep	every 15 min	auth-svc	Hard-delete expired + revoked sessions > 24h old	60s	Yes
listing-es-sync	every 30s	market-svc	Sync listings with es_doc_id=NULL to Elasticsearch	120s	Yes
reputation-recompute	daily 02:00 UTC	market-svc	Recompute reputation scores for all active users	4h	Yes
portfolio-health-batch	daily 03:00 UTC	portfolio-svc	Recompute health scores for all portfolios	2h	Yes
okr-period-close	daily 04:00 UTC	work-svc	Auto-score OKRs for periods ending today	1h	Yes
sprint-velocity-update	on sprint.closed event	wbs-svc	Update rolling 6-sprint velocity for WBS	30s	Yes
escrow-auto-release	every 5 min	exchange-svc	Release escrows past review_deadline	60s	Yes — idempotency key per escrow
campaign-status-sweep	every 30 min	fund-svc	Check campaign end dates; transition to FUNDED or FAILED	120s	Yes
proposal-vote-close	every 5 min	ao-svc	Close proposals past	60s	Yes

Job Name	Schedule	Service	Description	Timeout	Idempotent?
			voting_closes_at; compute outcome		
ai-recommendation-refresh	every 6h per user	ai-svc	Refresh stale recommendations for active users	varies	Yes
analytics-rollup	hourly	analytics-svc	Aggregate raw events into dashboard metrics	5m	Yes
tax-data-export	monthly (1st, 06:00 UTC)	exchange-svc	Generate 1099 data for users exceeding thresholds	1h	Yes
ai-session-cleanup	daily 05:00 UTC	ai-svc	Expire agent sessions idle > 2h	30m	Yes

13.2 Job Infrastructure

```
// All jobs use a distributed lock via Redis SETNX before execution.
// This prevents duplicate execution across multiple pods.

func RunJob(ctx context.Context, jobName string, fn func() error) error {
    lockKey := "job:lock:" + jobName
    lockTTL := 10 * time.Minute  // must exceed max job duration

    // Acquire distributed lock
    acquired, err := redis.SetNX(ctx, lockKey, instanceID, lockTTL)
    if err != nil || !acquired {
        return ErrJobAlreadyRunning  // another pod has the lock
    }
    defer redis.Del(ctx, lockKey)

    // Execute with timeout
    jobCtx, cancel := context.WithTimeout(ctx, jobTimeout[jobName])
    defer cancel()

    start := time.Now()
    if err := fn(); err != nil {
        metrics.JobFailure(jobName)
        logger.Error("job failed", "job", jobName, "duration", time.Since(start),
"err", err)
        alerting.Notify(jobName, err)  // PagerDuty on critical jobs
        return err
    }
    metrics.JobSuccess(jobName, time.Since(start))
```

```
    return nil
}
```

SECTION 14 · OBSERVABILITY

Observability & Logging Design

14.1 Structured Logging

```
// All services use structured JSON logging via zerolog.
// Log fields are consistent across all services for log aggregation.

// Standard log entry shape:
{
  "level":      "info",                    // debug|info|warn|error|fatal
  "ts":         "2024-01-15T10:30:00.123Z", // nanosecond precision
  "service":    "market-svc",
  "instance":   "pod-name",
  "trace_id":   "otel-trace-id",           // OpenTelemetry trace
  "span_id":    "otel-span-id",
  "request_id": "uuid",
  "user_id":    "uuid",                    // present for authenticated requests
  "event":      "order.placed",            // semantic event label
  "duration_ms": 145,                      // for request-scoped logs
  "msg":        "market order placed",
  // additional context fields as needed
}

// Security rule: PII (email, name, payment data) NEVER logged.
// Log user_id only; resolve to email in security review tooling only.
```

14.2 Metrics Catalog

Metric	Type	Labels	Alert Threshold
http_request_duration_seconds	Histogram	service, route, method, status_code	p95 > 500ms → warn; p99 > 2s → critical
http_request_total	Counter	service, route, method, status_code	error_rate > 1% → warn; > 5% → critical
event_bus_publish_total	Counter	service, event_type, status	failure_rate > 0.1% → warn
event_bus_consumer_lag	Gauge	service, consumer_group, subject	> 1000 unprocessed → warn; > 5000 → critical
db_query_duration_seconds	Histogram	service, query_name	p95 > 100ms → warn; p99 > 500ms → critical

Metric	Type	Labels	Alert Threshold
db_pool_connections_in_use	Gauge	service	> 80% of max → warn
cache_hit_rate	Gauge	service, cache_key_prefix	< 70% → warn (indicates TTL too short)
escrow_auto_release_count	Counter	outcome (released|refunded)	Daily refund_rate > 5% → investigate
ai_agent_tokens_used_total	Counter	user_id (sampled), provider, model	Monthly budget alert at 80% of LLM spend limit
job_duration_seconds	Histogram	job_name, status	duration > timeout × 0.8 → warn
active_websocket_connections	Gauge	service, room_type	> 50,000/pod → scale trigger
sprint_velocity_p50	Gauge	wbs_id (sampled)	Analytics only — no alert

14.3 Distributed Tracing

```
// All services instrument with OpenTelemetry SDK.
// Traces propagated via W3C Trace Context headers (traceparent, tracestate).
// Spans created for: HTTP handlers, gRPC calls, DB queries, NATS publish/consume.

// Critical trace attributes (set on all root spans):
trace.SetAttributes(
    attribute.String("user.id",          userID),
    attribute.String("workspace.id",      workspaceID),
    attribute.String("request.id",        requestID),
    attribute.String("service.version",   buildVersion),
    attribute.String("k8s.pod.name",      podName),
)

// Key trace flows to monitor in Grafana Tempo:
// 1. market_order_placed → escrow_created → escrow_funded (end-to-end checkout)
// 2. story_status_changed → okr_checkin_updated (OKR propagation latency)
// 3. campaign_funded → capital_pool_updated → distribution_computed (financial
flow)
// 4. ai_chat_request → llm_call → tool_executions → response_streamed (AI
latency)
```

🎯 **Key Design Invariants**

- Financial correctness: SELECT FOR UPDATE on all wallet operations; outbox pattern for all financial events; compensating entries only (no ledger updates or deletes)
- Idempotency everywhere: every mutation endpoint accepts Idempotency-Key; every background job is safely re-runnable

- State machine enforcement: all status transitions validated at service layer AND enforced by DB-level constraints (CHECK constraints on status columns)
- Soft deletes only: no hard deletes in domain tables during normal operation; 30-day recovery window
- Event-driven coupling: cross-module communication only via NATS events; no cross-service DB joins; no synchronous RPC for non-critical paths
- Privacy by default: all new resources default to private visibility; explicit opt-in required for any broader sharing
- AI transparency: every autonomous AI action logged in agent_action_log with full reasoning; users can review and override all AI actions
- Zero-trust networking: all inter-service communication via Istio mTLS; no cleartext internal traffic; pod identity via SPIFFE